

Semantic Reconciliation for Cross-Language Equivalence Verification: Proving $C \leftrightarrow \text{Rust}$ Migration Correctness Through Source-Level Semantic Analysis

Anonymous Authors
Under double-blind review

Abstract

Every C-to-Rust migration terminates with the same unanswerable question: *did we preserve the original behavior?* The state of practice—running the test suite and hoping—is fundamentally inadequate, because test suites encode a vanishing fraction of actual program semantics. Migration bugs escape to production because the source and target disagree on edge cases no test covers: a signed integer overflow that is undefined behavior in C but wraps or panics in Rust, a floating-point accumulation that rounds differently under C’s permissive extended-precision model versus Rust’s strict IEEE 754 semantics, a division by zero that silently corrupts memory in C but triggers a clean panic in Rust. These divergences are invisible to existing verification tools because the naïve approach—compiling both programs to LLVM IR and applying IR-level equivalence checkers—fails precisely because LLVM IR *erases* the semantic distinctions that cause cross-language divergence.

We present SEMREC, an equivalence verifier for structurally similar $C \leftrightarrow \text{Rust}$ function pairs. Given a C function and its purported Rust equivalent (produced by C2Rust or an LLM), SEMREC produces one of three verdicts: bounded equivalence (all paths within loop bound k verified by SMT), concrete divergence (a counterexample extracted from a Z3 model), or unknown (with quantified branch coverage). The key insight is a *semantic reconciliation framework* that preserves source-level semantic distinctions—integer overflow modes, floating-point precision models, and error handling conventions—through a shared typed SSA intermediate representation parameterized by a *semantic configuration* $\sigma = (\omega, \varphi, \varepsilon)$. A product program construction interleaves the two IR programs, sharing symbolic inputs and inserting *coercion points* at operations where C and Rust semantics diverge. We formalize the integer reconciliation algebra with overflow alignment contracts, define directed ε -approximation for floating-point tolerance with error accumulation tracking, and model C’s undefined behavior via angelic nondeterminism. We prove product program soundness, divergence witness correctness, three-output exhaustiveness, and coercion insertion completeness relative to a systematically enumerated divergence table. We evaluate SEMREC on 32 $C \leftrightarrow \text{Rust}$ function pairs spanning 10 divergence categories (arithmetic, overflow, control flow, bitwise, cast, error handling, floating point, loops, strings, and memory). On this benchmark, the tool proves equivalence for 16 pairs and produces concrete divergence witnesses for the remaining 16, with zero unknowns and zero errors. All counterexamples are extracted from Z3 satisfying models. Floating-point verification uses an **fp80** extended-precision model, and loop verification uses bounded symbolic execution with $k = 32$. The current benchmark is small-scale; scaling to whole-library analysis (e.g., libsodium, zlib) is future work.

1 Introduction

The C-to-Rust migration mandate has arrived with an urgency that outpaces tooling. The White House Office of the National Cyber Director [38], the Cybersecurity and Infrastructure Security Agency (CISA) [6], and the U.S. Department of Defense have issued directives urging migration of safety-critical C and C++ codebases to memory-safe languages, with Rust as the primary target. A 2023 Google analysis found that 70% of Android security vulnerabilities originate in C/C++ memory-safety bugs [14]—exactly the class of defects that Rust’s ownership discipline is designed to eliminate. Microsoft reports similar statistics for Windows kernel vulnerabilities [26]. The economic and security case for migration is overwhelming.

But migration is not merely syntactic transliteration. A transpiler like C2Rust [5] can mechanically translate C syntax into Rust syntax, producing compilable code saturated with `unsafe` blocks and no behavioral guarantee relative to the original. An LLM like GPT-4 can produce more idiomatic Rust, but studies find 15–30% of LLM-generated translations contain subtle behavioral divergences that pass human review [30]. The fundamental problem is that C and Rust—despite sharing an imperative computational model and compiling to the same LLVM backend—have *different semantics* for the same syntactic operations. Signed integer addition overflows to undefined behavior in C but wraps (release mode) or panics (debug mode) in Rust. A `float-to-int` cast that is out of range is undefined behavior in C but produces a saturated value in Rust (since Rust 1.45). An array access beyond bounds is silent undefined behavior in C but triggers a bounds-check panic in Rust.

Engineering organizations currently validate migrations through manual code review (error-prone, unscalable) and regression testing (covers only exercised paths). There is no tool today that can surface the exact inputs where a migration diverges.

The LLVM IR erasure thesis. The naïve approach to cross-language equivalence—compile both programs to LLVM IR, then apply existing equivalence checkers such as LLREVE [12], Alive2 [24], or UC-KLEE [32]—fails for a fundamental reason. LLVM IR *erases* the semantic distinctions that cause cross-language divergence.

Consider the expression `x + y` where `x` and `y` are signed 32-bit integers. When Clang compiles this from C, it emits `add nsw i32 %x, %y`—the `nsw` (no signed wrap) flag encodes C’s undefined-behavior-on-overflow as a poison value. When `rustc` compiles the same operation in release mode, it emits `add nsw i32 %x, %y`—the same instruction. In debug mode, `rustc` emits an explicit overflow check followed by a call to `core::panicking::panic`, but the core addition instruction is identical. At the LLVM IR level, both look similar or identical. Yet their source-language semantics differ fundamentally: C permits the compiler to assume overflow never happens (and optimize accordingly, potentially removing “dead” code that is actually reachable on overflow), while Rust guarantees either a defined wrapping result or a deterministic trap.

An IR-level equivalence checker will report these functions as equivalent—and be wrong. The programs diverge on the input ($x = \text{INT_MAX}, y = 1$): C has undefined behavior (the program may do anything, including returning an arbitrary value, crashing, or “optimizing” away surrounding code), while Rust produces either `-2147483648` (release wrap) or a panic (debug check). This is not a theoretical concern. It is the most common class of semantic divergence in C2Rust output, as we demonstrate in Section 7.

The same erasure affects floating-point semantics (C’s permissive extended precision versus Rust’s strict IEEE 754), error handling (C’s silent undefined behavior versus Rust’s `Result`/panic), memory safety (C’s unchecked pointer arithmetic versus Rust’s bounds-checked indexing), and type conversions (C’s implicit integer promotion versus Rust’s explicit typing). In every case, the

semantic distinction exists at the source level and is destroyed during lowering to LLVM IR.

Our approach. We propose SEMREC, a source-level semantic reconciliation framework for $C \leftrightarrow \text{Rust}$ equivalence verification. Rather than comparing programs through the lossy lens of a shared compilation target, SEMREC preserves source-level semantic distinctions by lowering both programs into a shared typed SSA intermediate representation whose operational semantics are *parameterized* by a semantic configuration $\sigma = (\omega, \varphi, \varepsilon)$ encoding overflow mode, floating-point precision model, and error model. The same IR program is interpreted differently under C’s configuration $\sigma_C = (\text{ub}, \text{extended}, \text{ub-silent})$ versus Rust’s $\sigma_R = (\text{wrap}|\text{panic}, \text{strict}, \text{trap}|\text{result})$. A product program construction interleaves the two IR programs, sharing symbolic inputs and inserting *coercion points*—semantic bridge assertions—at every operation where the divergence table T indicates a potential semantic gap. Bounded symbolic execution with SMT solving (Z3 [8]) discharges verification conditions, with differential fuzzing as a completeness fallback.

Contributions. This paper makes the following contributions:

1. **The LLVM IR erasure thesis**, demonstrated empirically: we exhibit concrete function pairs where IR-level equivalence checkers (Alive2, LLREVE) report “equivalent” and our tool reports “divergent” with a reproducible witness.
2. **A semantic reconciliation framework** for cross-language equivalence, including the semantic configuration space (Definition 3.1), the cross-language equivalence relation with angelic non-determinism for undefined behavior (Definition 3.15), the integer reconciliation algebra with overflow alignment contracts (Section 3.5), and directed ε -approximation with error accumulation for floating-point tolerance (Section 3.6).
3. **A product program construction with coercion insertion** (Section 4), proven sound (Theorem 5.2), with divergence witness correctness (Theorem 5.3), three-output exhaustiveness (Theorem 5.5), and coercion completeness (Theorem 5.6).
4. **An empirical evaluation** (Section 7) on 32 $C \leftrightarrow \text{Rust}$ function pairs spanning 10 divergence categories, demonstrating 16 equivalence proofs and 16 divergence detections with concrete counterexamples extracted from Z3 models.

Paper organization. Section 2 presents two detailed motivating examples. Section 3 develops the formal framework. Section 4 presents the algorithms with pseudocode and complexity analysis. Section 5 proves correctness. Section 6 describes the implementation. Section 7 presents the evaluation. Section 8 surveys related work. Section 9 discusses limitations and future work. Section 10 concludes.

2 Motivating Examples

We present two complete worked examples that illustrate the LLVM IR erasure problem and demonstrate SEMREC’s analysis. In each case, we show the C source, the corresponding Rust translation (as produced by C2Rust or an LLM), the LLVM IR (demonstrating erasure), and our tool’s step-by-step analysis leading to a concrete divergence witness.

2.1 Example 1: Signed Integer Overflow

Consider the following C function that computes a saturated sum, intended to return the sum of two integers if it fits in 32 bits, or `INT_MAX` if it would overflow:

```
1 #include <limits.h>
2 int safe_add(int a, int b) {
3     int sum = a + b; // UB if overflows!
4     if (sum < a)      // "overflow check" -- but UB already happened
5         return INT_MAX;
6     return sum;
7 }
```

Listing 1: C source: `safe_add`

The programmer intends the `if (sum < a)` check to detect overflow, but under C semantics, signed integer overflow is undefined behavior (ISO C11 §6.5/5). A conforming C compiler may assume `a + b` never overflows and optimize away the entire conditional, returning `sum` unconditionally. Indeed, GCC and Clang at `-O2` both eliminate the branch.

C2Rust produces the following Rust translation:

```
1 pub unsafe fn safe_add(a: i32, b: i32) -> i32 {
2     let sum: i32 = a.wrapping_add(b);
3     if sum < a {
4         return i32::MAX;
5     }
6     return sum;
7 }
```

Listing 2: C2Rust output: `safe_add`

C2Rust translates signed addition to `wrapping_add`, which is defined to wrap on overflow (two's complement). The overflow check `sum < a` now *works* in Rust: if `a = INT_MAX` and `b = 1`, then `sum = -2147483648` (wrapping), which is indeed less than `a`, so the function returns `INT_MAX`.

LLVM IR analysis. Compiling both to LLVM IR at `-O0`:

```
1 ; C version (clang -O0 -emit-llvm)
2 define i32 @safe_add(i32 %a, i32 %b) {
3     %sum = add nsw i32 %a, %b
4     %cmp = icmp slt i32 %sum, %a
5     br i1 %cmp, label %overflow, label %normal
6 overflow:
7     ret i32 2147483647
8 normal:
9     ret i32 %sum
10 }
11
12 ; Rust version (rustc --emit=llvm-ir -C opt-level=0)
13 define i32 @safe_add_rust(i32 %a, i32 %b) {
14     %sum = add i32 %a, %b ; no nsw flag!
15     %cmp = icmp slt i32 %sum, %a
16     br i1 %cmp, label %overflow, label %normal
17 overflow:
18     ret i32 2147483647
```

```

19 normal:
20     ret i32 %sum
21 }

```

Listing 3: LLVM IR for both functions (simplified)

The only difference is the `nsw` flag on the `add` instruction. LLREVE, operating on the IR structure, treats both as equivalent modulo the poison semantics of `nsw`—and reports “equivalent.” Alive2 similarly validates that the Rust version refines the C version (it produces defined behavior where C may produce poison). But this misses the critical divergence: on input ($a = 2147483647, b = 1$), the C function (compiled at `-O2`) returns -2147483648 (the compiler eliminated the overflow check), while the Rust function returns 2147483647 (the wrapping addition triggers the overflow branch).

SemRec analysis. Our tool proceeds as follows:

1. **Frontend lowering.** The C frontend (via `libclang`) lowers $a + b$ to $\text{add}_{\sigma_C}(a, b)$ with $\omega = \text{ub}$. The Rust frontend (via `syn`) lowers `a.wrapping_add(b)` to $\text{add}_{\sigma_R}(a, b)$ with $\omega = \text{wrap}$.
2. **Structural alignment.** The LCS-based alignment (Algorithm 3) matches the basic blocks by signature: both have an entry block with an add and comparison, a branch to overflow/normal blocks, and two return blocks. Alignment ratio: 100%.
3. **Product program construction.** The product program (Algorithm 4) shares symbolic inputs a, b and emits both additions with a coercion point:

$$\begin{aligned}
 c_sum &:= \text{add}_{\text{ub}}(a, b) \\
 r_sum &:= \text{add}_{\text{wrap}}(a, b) \\
 A_1 &\equiv \text{inRange}_{32}(a + b) \rightarrow (c_sum = r_sum)
 \end{aligned}$$

4. **SMT encoding.** The coercion assertion A_1 is encoded in QF_BV:

```

1  (declare-const a (_ BitVec 32))
2  (declare-const b (_ BitVec 32))
3  (define-fun exact () (_ BitVec 64)
4    (bvadd ((_ sign_extend 32) a) ((_ sign_extend 32) b)))
5  (define-fun in_range () Bool
6    (and (bvsge exact #xFFFFFFFF80000000)
7         (bvsle exact #x000000007FFFFFFF)))
8  (define-fun c_sum () (_ BitVec 32) ((_ extract 31 0) exact))
9  (define-fun r_sum () (_ BitVec 32) (bvadd a b))
10 ; Check: can in_range hold but c_sum != r_sum?
11 (assert (and in_range (not (= c_sum r_sum))))
12 (check-sat) ; UNSAT -- equivalent when defined

```

The solver returns UNSAT: when the mathematical sum is in range, both additions agree. But SEMREC also checks the complementary query—whether overflow is reachable:

```

1  (assert (not in_range))
2  (check-sat) ; SAT: a = 0x7FFFFFFF, b = 0x00000001

```

5. **Verdict.** The tool reports a *definedness divergence*: on input $(a = 2147483647, b = 1)$, C has undefined behavior while Rust produces a well-defined wrapping result of -2147483648 . The subsequent control flow diverges: C (optimized) skips the overflow check, while Rust enters the overflow branch and returns `INT_MAX`. The concrete witness is $(a = 2147483647, b = 1) \rightarrow \text{C: UB} \mid \text{Rust: } 2147483647$.

2.2 Example 2: Float-to-Int Saturation

Consider a C function that converts a double to an integer:

```
1 int float_to_int(double x) {
2     return (int)x; // UB if x is out of int range or NaN
3 }
```

Listing 4: C source: `float_to_int`

An LLM-generated Rust translation:

```
1 fn float_to_int(x: f64) -> i32 {
2     x as i32 // Saturating cast since Rust 1.45
3 }
```

Listing 5: LLM Rust translation: `float_to_int`

Since Rust 1.45, the `as` cast from floating-point to integer is *saturating*: values exceeding `i32::MAX` produce `i32::MAX`, values below `i32::MIN` produce `i32::MIN`, and NaN produces 0. In C, a cast from double to int where the value is outside the representable range is undefined behavior (ISO C11 §6.3.1.4).

Product program construction. The product program for this function pair is straightforward:

$$\begin{aligned} \text{c_result} &:= \text{fptosi}_{\text{ub}}(x) \\ \text{r_result} &:= \text{fptosi}_{\text{sat}}(x) \\ A_1 &\equiv \neg \text{isNaN}(x) \wedge (-2^{31} \leq x \leq 2^{31} - 1) \rightarrow (\text{c_result} = \text{r_result}) \end{aligned}$$

The coercion assertion A_1 says: if x is a finite value within the representable range of `i32`, then both casts produce the same truncated integer. Outside this range, C has UB while Rust saturates.

SMT encoding. Using the combined `QF_FP + QF_BV` theory:

```
1 (declare-const x Float64)
2 (define-fun out_of_range () Bool
3     (or (fp.isNaN x)
4         (fp.lt x (( _ to_fp 11 53) RNE (- 2147483648.0)))
5         (fp.gt x (( _ to_fp 11 53) RNE 2147483647.0))))
6 (assert out_of_range)
7 (check-sat) ; SAT: x = NaN
```

The solver immediately finds a divergence witness: $x = \text{NaN}$, for which C has undefined behavior and Rust produces 0. A second witness at $x = 2.2 \times 10^{10}$ demonstrates the saturation divergence: C is undefined, Rust returns `i32::MAX`.

These examples illustrate the two fundamental divergence categories our tool detects:

1. **Value divergence:** both programs have defined behavior, but produce different results (e.g., wrapping vs. no wrapping for in-range sums affected by earlier UB exploitation).
2. **Definedness divergence:** one program has undefined behavior (C) while the other has defined behavior (Rust wraps, saturates, or panics).

Both are invisible to IR-level analysis because the IR erases the semantic distinction between “undefined” and “defined-but-different.”

3 Formal Framework

We develop the formal underpinnings of SEMREC in six parts: the semantic configuration space that parameterizes language semantics (Section 3.1), source language fragments (Section 3.2), the shared typed SSA IR (Section 3.3), the cross-language reconciliation relation (Section 3.4), integer reconciliation (Section 3.5), floating-point reconciliation (Section 3.6), and error model reconciliation (Section 3.7).

3.1 Semantic Configuration Space

Definition 3.1 (Semantic Configuration). *A semantic configuration is a triple $\sigma = (\omega, \varphi, \varepsilon)$ drawn from the product domain*

$$\sigma \in \Sigma = \Omega \times \Phi \times E$$

where each component is defined as follows.

Definition 3.2 (Overflow Mode). *The overflow mode domain is*

$$\Omega = \{\text{ub}, \text{wrap}, \text{panic}, \text{sat}\}$$

with interpretations:

- **ub** (*undefined behavior*): *The result of an overflowing operation is unconstrained; the program’s subsequent execution has no defined semantics. This is C’s model for signed integer overflow per ISO C11 §6.5/5.*
- **wrap** (*two’s complement wrapping*): *The result is computed modulo 2^w where w is the bit-width. This is Rust’s release-mode semantics and C’s unsigned arithmetic.*
- **panic** (*checked trap*): *The operation checks for overflow and halts execution with a trap if overflow occurs. This is Rust’s debug-mode semantics.*
- **sat** (*saturating*): *The result is clamped to $[\text{MIN}_w, \text{MAX}_w]$. This is Rust’s explicit `saturating_add` family.*

The C configuration uses $\omega_C = \text{ub}$ for signed arithmetic and $\omega_C = \text{wrap}$ for unsigned. The Rust configuration uses $\omega_R \in \{\text{wrap}, \text{panic}\}$ depending on compilation mode, with `sat` available via explicit API.

Definition 3.3 (Float Model). *The floating-point precision model domain is*

$$\Phi = \{\text{extended}, \text{strict}\}$$

where:

- *extended*: The implementation may evaluate floating-point expressions at higher precision than the declared type (e.g., 80-bit x87 extended precision for `double` operations). Intermediate results may retain extra precision across expression boundaries. This is permitted by ISO C11 §5.2.4.2.2 with `FLT_EVAL_METHOD` $\in \{1, 2\}$. The result of an expression e of type τ is some value v such that $v = \text{round}_\tau(\text{eval}_{\tau'}(e))$ for some $\tau' \supseteq \tau$, where τ' is chosen nondeterministically.
- *strict*: Every floating-point operation produces a result rounded to the declared type's precision at every operation boundary, per IEEE 754-2019. This is Rust's model.

Definition 3.4 (Error Model). *The error model domain is*

$$E = \{\text{ub-silent}, \text{trap}, \text{result}\}$$

where:

- *ub-silent*: Erroneous conditions (null dereference, division by zero, out-of-bounds access) yield undefined behavior. This is C's model.
- *trap*: Erroneous conditions cause an immediate, observable program halt (panic). This is Rust's model for bounds checks, debug-mode overflow, and `unwrap()` on `None/Err`.
- *result*: Erroneous conditions are encoded in the return type as `Result<T, E>` or `Option<T>`. This is idiomatic Rust's model for recoverable errors.

The C configuration is $\sigma_C = (\text{ub}, \text{extended}, \text{ub-silent})$. The Rust configuration is $\sigma_R = (\text{wrap|panic}, \text{strict}, \text{trap|result})$.

Definition 3.5 (Configuration Divergence Points). *Given two configurations $\sigma_1 = (\omega_1, \varphi_1, \varepsilon_1)$ and $\sigma_2 = (\omega_2, \varphi_2, \varepsilon_2)$, the divergence set is*

$$\Delta(\sigma_1, \sigma_2) = \{d \in \{\text{overflow}, \text{float}, \text{error}\} \mid \pi_d(\sigma_1) \neq \pi_d(\sigma_2)\}$$

where π_d is the projection onto the d -th component. For the canonical $C \leftrightarrow \text{Rust}$ pair, $\Delta(\sigma_C, \sigma_R) = \{\text{overflow}, \text{float}, \text{error}\}$ —all three components diverge.

3.2 Source Language Fragments

We define source language fragments as structured operational semantics over typed expressions. These capture the C2Rust-relevant subset, not full language models.

Definition 3.6 (C Fragment Syntax).

$$\begin{aligned} \tau_C &::= \text{int}_{w,s} \mid \text{float}_p \mid \text{ptr}(\tau_C) \mid \text{void} \mid \text{struct}\{l_i : \tau_i\} \\ e_C &::= x \mid n \mid e_1 \oplus e_2 \mid e_1 \otimes e_2 \mid *e \mid \&e \mid e_1[e_2] \\ &\quad \mid \text{if } e_1 \text{ then } e_2 \text{ else } e_3 \mid \text{while } e_1 \text{ do } e_2 \mid \text{let } x : \tau = e_1 \text{ in } e_2 \\ &\quad \mid f(e_1, \dots, e_n) \mid (\tau)e \mid e.l \\ \oplus &::= + \mid - \mid \ll \mid \gg \mid \& \mid \mid \mid \wedge \\ \otimes &::= * \mid / \end{aligned}$$

where $\text{int}_{w,s}$ is a fixed-width integer of width $w \in \{8, 16, 32, 64, 128\}$ and signedness $s \in \{\text{signed}, \text{unsigned}\}$, float_p has precision $p \in \{f32, f64\}$, and $\otimes$ is distinguished from $\oplus$ for division-by-zero semantics.

Definition 3.7 (C Fragment Operational Semantics). *The small-step relation $\langle e, M, \sigma_C \rangle \rightarrow_C \langle e', M', \sigma_C \rangle \mid \perp_{UB}$ is defined by the following rules.*

(C-Add-Signed): For $e_1 \oplus e_2$ where $\oplus$ is addition on $\text{int}_{w,\text{signed}}$:

$$\frac{\langle e_1, M \rangle \Downarrow v_1 \quad \langle e_2, M \rangle \Downarrow v_2 \quad v_1 + v_2 \in [-2^{w-1}, 2^{w-1} - 1]}{\langle e_1 + e_2, M, \sigma_C \rangle \rightarrow_C \langle v_1 + v_2, M, \sigma_C \rangle}$$

$$\frac{\langle e_1, M \rangle \Downarrow v_1 \quad \langle e_2, M \rangle \Downarrow v_2 \quad v_1 + v_2 \notin [-2^{w-1}, 2^{w-1} - 1]}{\langle e_1 + e_2, M, \sigma_C \rangle \rightarrow_C \perp_{UB}}$$

(C-Add-Unsigned): For unsigned $\text{int}_{w,\text{unsigned}}$:

$$\frac{\langle e_1, M \rangle \Downarrow v_1 \quad \langle e_2, M \rangle \Downarrow v_2}{\langle e_1 + e_2, M, \sigma_C \rangle \rightarrow_C \langle (v_1 + v_2) \bmod 2^w, M, \sigma_C \rangle}$$

(C-Float-Op): For floating-point operations under $\varphi = \text{extended}$:

$$\frac{\langle e_1, M \rangle \Downarrow v_1 \quad \langle e_2, M \rangle \Downarrow v_2 \quad p' \geq p}{\langle e_1 \oplus_p e_2, M, \sigma_C \rangle \rightarrow_C \langle \text{round}_p(\text{eval}_{p'}(v_1 \oplus v_2)), M, \sigma_C \rangle}$$

where p' is chosen nondeterministically from $\{p, \dots, \text{extended-80}\}$, reflecting C 's `FLT_EVAL_METHOD` freedom.

(C-UB-Angelic): When an operation triggers undefined behavior, the result is existentially quantified over all possible bitvector values of the appropriate type:

$$\frac{\langle e_1, M \rangle \Downarrow v_1 \quad \langle e_2, M \rangle \Downarrow v_2 \quad v_1 + v_2 \notin R_w}{\langle e_1 + e_2, M, \sigma_C \rangle \rightarrow_C \langle \exists r. r, M, \sigma_C \rangle}$$

Under angelic nondeterminism, the C program can produce any value at a UB point. This models what a conforming C compiler could do, making equivalence easier to achieve, so a failure of equivalence under angelic semantics is a strong result.

Definition 3.8 (Rust Fragment Syntax).

$$\begin{aligned} \tau_R &::= \text{int}_{w,s} \mid \text{float}_p \mid \&\tau_R \mid \&\text{mut } \tau_R \mid \text{Box} < \tau_R > \\ &\mid (\tau_1, \dots, \tau_n) \mid \text{struct}\{l_i : \tau_i\} \mid \text{Result} < \tau, E > \mid \text{Option} < \tau > \\ e_R &::= x \mid n \mid e_1 \oplus_{\text{mode}} e_2 \mid *e \mid \&e \mid \&\text{mut } e \mid e_1[e_2] \\ &\mid \text{if } e_1 \{e_2\} \text{ else } \{e_3\} \mid \text{loop } \{e\} \mid \text{let } x : \tau = e_1; e_2 \\ &\mid f(e_1, \dots, e_n) \mid e \text{ as } \tau \mid e.l \mid e? \mid \text{match } e \{p_i \Rightarrow e_i\} \\ \text{mode} &::= \text{checked} \mid \text{wrapping} \mid \text{saturating} \end{aligned}$$

Definition 3.9 (Rust Fragment Operational Semantics). *The small-step relation $\langle e, M, \sigma_R \rangle \rightarrow_R \langle e', M', \sigma_R \rangle \mid \perp_{\text{panic}}$ is defined by:*

(R-Add-Wrap): Under $\omega = \text{wrap}$:

$$\frac{\langle e_1, M \rangle \Downarrow v_1 \quad \langle e_2, M \rangle \Downarrow v_2}{\langle e_1 +_{\text{wrap}} e_2, M, \sigma_R \rangle \rightarrow_R \langle \text{trunc}_w(v_1 + v_2), M, \sigma_R \rangle}$$

where $\text{trunc}_w(v) = ((v + 2^{w-1}) \bmod 2^w) - 2^{w-1}$ for signed types.

(R-Add-Checked): Under $\omega = \text{panic}$:

$$\frac{\langle e_1, M \rangle \Downarrow v_1 \quad \langle e_2, M \rangle \Downarrow v_2 \quad v_1 + v_2 \in R_w}{\langle e_1 +_{\text{checked}} e_2, M, \sigma_R \rangle \rightarrow_R \langle v_1 + v_2, M, \sigma_R \rangle}$$

$$\frac{\langle e_1, M \rangle \Downarrow v_1 \quad \langle e_2, M \rangle \Downarrow v_2 \quad v_1 + v_2 \notin R_w}{\langle e_1 +_{\text{checked}} e_2, M, \sigma_R \rangle \rightarrow_R \perp_{\text{panic}}}$$

(R-Float-Op): Under $\varphi = \text{strict}$:

$$\frac{\langle e_1, M \rangle \Downarrow v_1 \quad \langle e_2, M \rangle \Downarrow v_2}{\langle e_1 \oplus_p e_2, M, \sigma_R \rangle \rightarrow_R \langle \text{round}_p(v_1 \oplus v_2), M, \sigma_R \rangle}$$

No nondeterminism: the precision is exactly p .

3.3 Shared Typed SSA IR

Definition 3.10 (IR Syntax). *The shared IR is a typed SSA language parameterized by σ :*

$$\begin{aligned} I ::= & x := \text{op}(y_1, \dots, y_n) \mid x := \text{load}(p) \mid \text{store}(p, v) \\ & \mid \text{br}(c, L_t, L_f) \mid \text{jmp}(L) \mid \text{ret}(v) \\ & \mid x := \text{call}(f, y_1, \dots, y_n) \mid \text{assert_coerce}(c, \text{tag}) \\ \text{op} ::= & \text{add}_\sigma \mid \text{sub}_\sigma \mid \text{mul}_\sigma \mid \text{div}_\sigma \mid \text{rem}_\sigma \\ & \mid \text{fadd}_\sigma \mid \text{fsub}_\sigma \mid \text{fmul}_\sigma \mid \text{fdiv}_\sigma \\ & \mid \text{shl} \mid \text{lshr} \mid \text{ashr} \mid \text{and} \mid \text{or} \mid \text{xor} \\ & \mid \text{icmp}_{cc} \mid \text{fcmp}_{cc} \\ & \mid \text{zext}_w \mid \text{sxt}_w \mid \text{trunc}_w \mid \text{bitcast} \mid \text{inttoptr} \mid \text{ptrtoint} \\ B ::= & L : I_1; I_2; \dots; I_n; t \\ F ::= & \text{fun } f(x_1 : \tau_1, \dots, x_n : \tau_n) \rightarrow \tau \{B_1, \dots, B_m\} \end{aligned}$$

The subscript σ on arithmetic operations is the key design feature: add_σ is interpreted differently depending on the semantic configuration.

Definition 3.11 (IR Typing Rules). *Selected rules (judgments of the form $\Gamma \vdash_\sigma I : \text{ok}$):*

(T-Add):

$$\frac{\Gamma(y_1) = \text{int}_{w,s} \quad \Gamma(y_2) = \text{int}_{w,s}}{\Gamma \vdash_\sigma x := \text{add}_\sigma(y_1, y_2) : \text{int}_{w,s}}$$

(T-FAdd):

$$\frac{\Gamma(y_1) = \text{float}_p \quad \Gamma(y_2) = \text{float}_p}{\Gamma \vdash_\sigma x := \text{fadd}_\sigma(y_1, y_2) : \text{float}_p}$$

(T-Coerce):

$$\frac{\Gamma \vdash c : \text{bool} \quad \text{tag} \in \text{CoercionTag}}{\Gamma \vdash_\sigma \text{assert_coerce}(c, \text{tag}) : \text{ok}}$$

Definition 3.12 (IR Operational Semantics). *The semantics $\llbracket \cdot \rrbracket_\sigma$ maps each instruction to a state transformer. The critical case for addition:*

$$\llbracket x := \text{add}_\sigma(y_1, y_2) \rrbracket_\sigma(S) = \begin{cases} S[x \mapsto v_1 + v_2] & \text{if } \omega(\sigma) = \text{ub} \wedge v_1 + v_2 \in R_w \\ \perp_{\text{UB}} & \text{if } \omega(\sigma) = \text{ub} \wedge v_1 + v_2 \notin R_w \\ S[x \mapsto \text{trunc}_w(v_1 + v_2)] & \text{if } \omega(\sigma) = \text{wrap} \\ \perp_{\text{panic}} & \text{if } \omega(\sigma) = \text{panic} \wedge v_1 + v_2 \notin R_w \\ S[x \mapsto v_1 + v_2] & \text{if } \omega(\sigma) = \text{panic} \wedge v_1 + v_2 \in R_w \\ S[x \mapsto \text{clamp}(v_1 + v_2, R_w)] & \text{if } \omega(\sigma) = \text{sat} \end{cases}$$

where $R_w = [-2^{w-1}, 2^{w-1} - 1]$ for signed types, $[0, 2^w - 1]$ for unsigned, and $S[x \mapsto v]$ denotes state update with $v_1 = S(y_1)$, $v_2 = S(y_2)$.

3.4 Cross-Language Reconciliation Relation

Definition 3.13 (Observable Output). *For a function f with return type τ , the observable output on input $\bar{t}$ under configuration σ is:*

$$\text{obs}_\sigma(f, \bar{t}) = \begin{cases} \text{Val}(v) & \text{if } \llbracket f \rrbracket_\sigma(\bar{t}) = v \\ \text{UB} & \text{if } \llbracket f \rrbracket_\sigma(\bar{t}) = \perp_{\text{UB}} \\ \text{Panic} & \text{if } \llbracket f \rrbracket_\sigma(\bar{t}) = \perp_{\text{panic}} \\ \text{Err}(e) & \text{if } \llbracket f \rrbracket_\sigma(\bar{t}) = \text{Err}(e) \end{cases}$$

Definition 3.14 (Well-Defined Domain). *The well-defined domain of f under σ is:*

$$\mathcal{D}_\sigma(f) = \{\bar{t} \mid \text{obs}_\sigma(f, \bar{t}) = \text{Val}(v) \text{ for some } v\}$$

Definition 3.15 (Cross-Language Equivalence with Angelic UB). *Two functions f_C (under σ_C) and f_R (under σ_R) are equivalent modulo semantic configuration, written $f_C \approx_{\sigma_C, \sigma_R} f_R$, if and only if:*

$$\forall \bar{t} \in \mathcal{D}_{\sigma_R}(f_R). \exists \text{ub_choices}. \text{obs}_{\sigma_C}^{\text{angelic}}(f_C, \bar{t}, \text{ub_choices}) \sim_\tau \text{obs}_{\sigma_R}(f_R, \bar{t})$$

where $\sim_\tau$ is type-indexed output equivalence:

- For $\text{int}_{w,s}$: $v_1 \sim v_2$ iff $v_1 = v_2$.
- For float_p : $v_1 \sim v_2$ iff $|v_1 - v_2| \leq \max(\varepsilon \cdot |v_1|, \delta)$ (see Definition 3.31).
- For $\text{ptr}(\tau)$: structural equality of pointed-to values.
- For struct types: component-wise $\sim$.

The quantification is over Rust's well-defined domain $\mathcal{D}_{\sigma_R}(f_R)$, since Rust (in safe code) has no UB, so $\mathcal{D}_{\sigma_R}$ is typically the full input space. For each input, we ask whether some resolution of C's UB agrees with Rust.

Definition 3.16 (Semantic Divergence Witness). *A divergence witness for the pair (f_C, f_R) is an input $\bar{t}$ such that exactly one of the following holds:*

1. **Value divergence:** $\bar{t} \in \mathcal{D}_{\sigma_C}(f_C) \cap \mathcal{D}_{\sigma_R}(f_R)$ and $\text{obs}_{\sigma_C}(f_C, \bar{t}) \not\sim_\tau \text{obs}_{\sigma_R}(f_R, \bar{t})$.
2. **Definedness divergence:** $\bar{t} \in \mathcal{D}_{\sigma_R}(f_R) \setminus \mathcal{D}_{\sigma_C}(f_C)$ and $\forall \text{ub_choices}. \text{obs}_{\sigma_C}^{\text{angelic}}(f_C, \bar{t}, \text{ub_choices}) \not\sim_\tau \text{obs}_{\sigma_R}(f_R, \bar{t})$.

Case (2) captures the critical scenario: an input where C has undefined behavior and even the most favorable angelic resolution of UB cannot make C agree with Rust.

3.5 Integer Reconciliation

This section constitutes the primary formal contribution: a complete algebra for reconciling integer arithmetic across language boundaries.

Definition 3.17 (Bitvector Type). *$\text{BV}(w, s)$ denotes the bitvector type of width $w \in \mathbb{N}^+$ and signedness $s \in \{\mathbf{u}, \mathbf{s}\}$. The value domain is:*

$$\begin{aligned} \text{Val}(\text{BV}(w, \mathbf{u})) &= \{0, 1, \dots, 2^w - 1\} \\ \text{Val}(\text{BV}(w, \mathbf{s})) &= \{-2^{w-1}, \dots, 2^{w-1} - 1\} \end{aligned}$$

Definition 3.18 (Embedding Map). *The embedding $\text{embed} : \text{BV}(w, s) \rightarrow \mathbb{Z}$ maps bitvector values to their mathematical integer interpretation:*

$$\begin{aligned} \text{embed}(v, w, \mathbf{u}) &= v \\ \text{embed}(v, w, \mathbf{s}) &= \begin{cases} v & \text{if } v < 2^{w-1} \\ v - 2^w & \text{if } v \geq 2^{w-1} \end{cases} \end{aligned}$$

Definition 3.19 (Truncation Map). *The truncation $\text{trunc} : \mathbb{Z} \rightarrow \text{BV}(w, s)$ maps mathematical integers to bitvector values:*

$$\begin{aligned} \text{trunc}(z, w, \mathbf{u}) &= z \bmod 2^w \\ \text{trunc}(z, w, \mathbf{s}) &= \begin{cases} z & \text{if } z \in \text{Val}(\text{BV}(w, \mathbf{s})) \\ ((z + 2^{w-1}) \bmod 2^w) - 2^{w-1} & \text{otherwise} \end{cases} \end{aligned}$$

Lemma 3.20 (Roundtrip). *For all $v \in \text{Val}(\text{BV}(w, s))$: $\text{trunc}(\text{embed}(v, w, \mathbf{s}), w, s) = v$.*

Proof. For unsigned: $\text{embed}(v, w, \mathbf{u}) = v$ and $\text{trunc}(v, w, \mathbf{u}) = v \bmod 2^w = v$ since $0 \leq v < 2^w$.

For signed with $v \geq 0$ (i.e., $v < 2^{w-1}$): $\text{embed}(v, w, \mathbf{s}) = v$, and $v \in \text{Val}(\text{BV}(w, \mathbf{s}))$, so $\text{trunc}(v, w, \mathbf{s}) = v$.

For signed with $v < 0$ (i.e., the raw bit pattern $v' \geq 2^{w-1}$): $\text{embed}(v', w, \mathbf{s}) = v' - 2^w = v$. Then $v \in [-2^{w-1}, -1] \subseteq \text{Val}(\text{BV}(w, \mathbf{s}))$, so $\text{trunc}(v, w, \mathbf{s}) = v$. $\square$

Lemma 3.21 (Truncation is Wrapping). *For all $z \in \mathbb{Z}$: $\text{trunc}(z, w, s)$ is the unique value $v \in \text{Val}(\text{BV}(w, s))$ such that $v \equiv z \pmod{2^w}$.*

Proof. For unsigned: $\text{trunc}(z, w, \mathbf{u}) = z \bmod 2^w$, which is by definition the unique representative in $[0, 2^w - 1]$ congruent to z modulo 2^w .

For signed: Let $v = ((z + 2^{w-1}) \bmod 2^w) - 2^{w-1}$. Then $v + 2^{w-1} = (z + 2^{w-1}) \bmod 2^w \in [0, 2^w - 1]$, so $v \in [-2^{w-1}, 2^{w-1} - 1] = \text{Val}(\text{BV}(w, \mathbf{s}))$. Moreover, $v = (z + 2^{w-1}) \bmod 2^w - 2^{w-1} \equiv z + 2^{w-1} - 2^{w-1} = z \pmod{2^w}$. $\square$

Definition 3.22 (Overflow Alignment Contract). *An overflow alignment contract for an arithmetic operation $\oplus$ on $\text{BV}(w, s)$ under configuration pair (ω_1, ω_2) is a formula $\text{OAC}_{\omega_1, \omega_2}(a, b, r_1, r_2)$ specifying the verification obligation relating the results r_1 (under ω_1) and r_2 (under ω_2). Let $z = \text{embed}(a) \oplus_{\mathbb{Z}} \text{embed}(b)$ denote the mathematical result.*

We now define the contracts for each pair relevant to C $\leftrightarrow$ Rust verification.

Definition 3.23 (OAC: ub $\times$ wrap). *C signed $\times$ Rust release mode:*

$$\text{OAC}_{\text{ub}, \text{wrap}}(a, b, r_C, r_R) \equiv (z \in \text{Val}(\text{BV}(w, s))) \rightarrow (r_C = r_R)$$

When z is in range, both produce z and must agree. When z is out of range, C has UB (so no obligation under angelic semantics), but the system reports a definedness divergence as a diagnostic. The Rust result $\text{trunc}(z)$ is well-defined but the C result is unconstrained.

The SMT encoding for this contract (width 32, addition) is:

```

1 (declare-const a (_ BitVec 32))
2 (declare-const b (_ BitVec 32))
3 (define-fun z () (_ BitVec 64)
4   (bvadd ((_ sign_extend 32) a) ((_ sign_extend 32) b)))
5 (define-fun in_range () Bool
6   (and (bvsge z #xFFFFFFFF80000000)
7        (bvsle z #x000000007FFFFFFF)))
8 (define-fun r_C () (_ BitVec 32) ((_ extract 31 0) z))
9 (define-fun r_R () (_ BitVec 32) (bvadd a b))
10 (assert (not (=> in_range (= r_C r_R))))
11 (check-sat) ; UNSAT means equivalent on defined inputs

```

Listing 6: SMT encoding for $\text{OAC}_{\text{ub,wrap}}$

Definition 3.24 ($\text{OAC: ub} \times \text{panic}$). C signed $\times$ Rust debug mode:

$$\text{OAC}_{\text{ub,panic}}(a, b, r_C, r_R) \equiv (z \in R_w) \rightarrow (r_C = r_R)$$

When $z \in R_w$, both produce z and must agree. When $z \notin R_w$, C has UB and Rust panics—a definedness divergence with classification “C-UB, Rust-panic.” The system generates a secondary witness for this case by checking satisfiability of $\neg \text{inRange}_w(y_1 + y_2)$.

Definition 3.25 ($\text{OAC: ub} \times \text{sat}$). C signed $\times$ Rust saturating:

$$\text{OAC}_{\text{ub,sat}}(a, b, r_C, r_R) \equiv (z \in R_w) \rightarrow (r_C = r_R)$$

When $z \notin R_w$, C has UB while Rust saturates to the boundary. This is always a divergence on overflow inputs since saturation produces a specific value and C is unconstrained.

Definition 3.26 (Division Alignment Contract). For signed division a/b :

$$\text{OAC}_{\text{div}}(a, b, r_C, r_R) \equiv (b \neq 0 \wedge \neg(a = \text{MIN}_w \wedge b = -1)) \rightarrow (r_C = r_R)$$

Three divergence cases: (1) $b = 0$: UB in C , panic in Rust. (2) $a = \text{MIN}_w, b = -1$: signed overflow (result would be $\text{MAX}_w + 1$), UB in C , panic in Rust. (3) Otherwise: both compute truncation-toward-zero division identically (ISO C11 §6.5.5, Rust Reference).

Definition 3.27 (Shift Alignment Contract). For left shift $a \ll b$:

$$\text{OAC}_{\text{shl}}(a, b, r_C, r_R) \equiv (0 \leq b < w) \rightarrow (r_C = r_R)$$

C has UB when shifting by $\geq w$ or shifting a negative signed value left. Rust masks the shift amount: the effective shift is $b \bmod w$ (wrapping mode) or panics if $b \geq w$ (checked mode).

Definition 3.28 (Integer Promotion Contract). C performs integer promotion on operands narrower than `int` (typically 32 bits). For a $\text{BV}(w, s)$ operation in C with $w < 32$, the operands are promoted to $\text{BV}(32, s)$ before the operation, and the result is truncated back. Rust performs no implicit promotion. Let $a, b \in \text{BV}(w, s)$ with $w < 32$. In C :

$$r_C = \text{trunc}_w(\text{sext}_{32}(a) \oplus_{32} \text{sext}_{32}(b))$$

In Rust:

$$r_R = a \oplus_w b$$

The coercion assertion is $A_{\text{promo}} \equiv (r_C = r_R)$, checked directly since the promotion is part of C ’s defined semantics (not UB).

3.6 Floating-Point Reconciliation

Definition 3.29 (Strict Float Model). *Under $\varphi = \text{strict}$, every operation $\oplus$ on float_p produces:*

$$\text{eval}_{\text{strict}}(a \oplus b, p) = \text{round}_p(a \oplus_{\mathbb{R}} b)$$

where round_p is the IEEE 754 *roundTiesToEven* operation for precision p , and $\oplus_{\mathbb{R}}$ denotes the exact real-valued operation.

Definition 3.30 (Extended Float Model). *Under $\varphi = \text{extended}$, the result is:*

$$\text{eval}_{\text{ext}}(a \oplus b, p) = \text{round}_p(\text{round}_{p'}(a \oplus_{\mathbb{R}} b))$$

where $p' \geq p$ is chosen nondeterministically. The outer round_p occurs at expression boundaries (assignment, cast, function call) per C11 §5.2.4.2.2. The nondeterminism in p' means that C float computations denote sets of possible values.

Definition 3.31 (Directed ε -Approximation). *Given a reference computation f_{ref} (C under abstract machine semantics) and an approximation f_{approx} (Rust under strict IEEE 754), f_{approx} is an ε -approximation of f_{ref} , written $f_{\text{ref}} \sqsubseteq_{\varepsilon, \delta} f_{\text{approx}}$, if for all inputs $\bar{t} \in \mathcal{D}(f_{\text{ref}})$:*

$$|\text{obs}(f_{\text{ref}}, \bar{t}) - \text{obs}(f_{\text{approx}}, \bar{t})| \leq \max(\varepsilon \cdot |\text{obs}(f_{\text{ref}}, \bar{t})|, \delta)$$

Default parameters: $\varepsilon = 2^{-50}$ (approximately 1 ULP for double-precision results computed via extended precision), $\delta = 2^{-1074}$ (the smallest positive denormalized double).

This is not an equivalence relation and we do not claim it is. It is a directed approximation bound. Transitivity is replaced by the triangle inequality: if $f_1 \sqsubseteq_{\varepsilon_1} f_2$ and $f_2 \sqsubseteq_{\varepsilon_2} f_3$, then $f_1 \sqsubseteq_{\varepsilon_1 + \varepsilon_2 + \varepsilon_1 \varepsilon_2} f_3$.

Definition 3.32 (Error Accumulation). *At each coercion point k in the product program, the accumulated tolerance bound is:*

$$\varepsilon_k = \varepsilon_{k-1} + \varepsilon_{\text{op}} + \varepsilon_{k-1} \cdot \varepsilon_{\text{op}}$$

where ε_{op} is the single-operation tolerance. The final output comparison uses $\varepsilon_{\text{final}}$, not ε per operation. The SMT encoding at the output return point uses the accumulated bound.

Definition 3.33 (Genuine vs. Benign FP Divergence). *A floating-point divergence on input $\bar{t}$ producing values v_C, v_R is:*

- **Benign** if $|v_C - v_R| \leq \max(\varepsilon \cdot \max(|v_C|, |v_R|), \delta)$: the difference is within the tolerance expected from extended-precision nondeterminism.
- **Genuine** if the inequality is violated: the difference exceeds what extended-precision nondeterminism can explain.

3.7 Error Model Reconciliation

Definition 3.34 (Error Outcome Domain). *The error outcome domain for a single operation is:*

$$\text{Outcome} = \text{Val}(v) \mid \text{UB} \mid \text{Panic}(msg) \mid \text{Err}(e) \mid \text{Undef}$$

Definition 3.35 (Error Reconciliation). *The reconciliation maps every $(\text{Outcome}_C, \text{Outcome}_R)$ pair to a verdict, as shown in Table 1.*

Table 1: Error reconciliation table. Each cell shows the verdict and the coercion assertion generated.

C Outcome	Rust Outcome	Verdict	Coercion
Val(v_1)	Val(v_2)	Check $v_1 \sim_\tau v_2$	Standard output equality
UB	Panic	Definedness div.	“C UB, Rust panics”
UB	Err(e)	Definedness div.	“C UB, Rust returns error”
UB	Val(v)	Definedness div.	“C UB, Rust succeeds”
Val(v)	Panic	Definedness div.	“C succeeds, Rust panics”
Val(v)	Err(e)	Error handling div.	“C silent, Rust signals error”

Table 2: Semantic divergence table T for $C \leftrightarrow \text{Rust}$. Every entry generates a coercion point in the product program.

OpKind	Component	σ_C	σ_R	Nature
add/sub/mul (signed)	overflow	ub	wrap	Value divergence on overflow
add/sub/mul (signed)	overflow	ub	panic	Definedness divergence
div/rem (signed)	overflow	ub	panic	MIN/ -1 and $\div 0$
shl/shr	overflow	ub	wrap/panic	Shift $\geq$ width
fadd/fsub/fmul/fdiv	float	extended	strict	Precision divergence
fptosi	float/error	ub	sat	Out-of-range saturation
array index	error	ub-silent	trap	Bounds check
ptr deref	error	ub-silent	trap	Null check
int $\rightarrow$ ptr cast	error	ub-silent	trap	Provenance
transmute/union	type-punning	impl-def	impl-def	Representation divergence

Definition 3.36 (Error Coercion Assertion). *At a program point where C may have UB and Rust may panic, the coercion assertion is:*

$$A_{\text{err}} \equiv \text{defined}_C(\bar{l}) \rightarrow \text{defined}_R(\bar{l}) \rightarrow (\text{val}_C = \text{val}_R)$$

where $\text{defined}_C(\bar{l})$ encodes the precondition for C ’s operation to be defined (e.g., pointer is non-null, index is in bounds, divisor is non-zero), and $\text{defined}_R(\bar{l})$ encodes the precondition for Rust’s operation to not panic.

Definition 3.37 (Semantic Divergence Table). *The semantic divergence table T is a finite relation $T \subseteq \text{opkind} \times \text{Component} \times (\Sigma \times \Sigma)$ enumerating, for each IR operation kind, the semantic configuration component along which it may diverge. Table 2 presents T for the $C \leftrightarrow \text{Rust}$ pair.*

Definition 3.38 (Coercion Point). *Given IR programs P_C, P_R and the divergence table T , a coercion point is a triple (pc_C, pc_R, A) where pc_C is a program counter in P_C at an instruction I_C , pc_R is the structurally aligned program counter in P_R at instruction I_R , and A is a coercion assertion: a boolean formula over the symbolic state, defined by case analysis on the divergence entry as specified in Definitions 3.23–3.36.*

Definition 3.39 (Coercion Insertion Function). *The function $\text{InsertCoercions} : \text{IR} \times \text{IR} \times T \rightarrow \text{IR}^\times$ takes two aligned IR programs and the divergence table and produces a product program with coercion*

assertions inserted at every divergence point:

$\text{InsertCoercions}(P_C, P_R, T) = P^\times$ such that for every aligned pair (I_C^i, I_R^i)

where $\text{opkind}(I_C^i) \in \pi_1(T)$, the assertion A_i is inserted immediately after both instructions.

4 Algorithms

We present the seven core algorithms of SEMREC, each with pseudocode in the `algorithm2e` environment and complexity analysis.

4.1 AST-to-IR Lowering

Algorithm 1: C AST-to-IR Lowering

Input : Clang AST *ast*, target platform info *target*
Output: IR function *F* with $\sigma = (\text{ub}, \text{extended}, \text{ub-silent})$

- 1 $F \leftarrow \text{new Function}(ast.name);$
- 2 $F.\sigma \leftarrow (\text{ub}, \text{extended}, \text{ub-silent});$
- 3 $promo \leftarrow \text{ComputeCPromotions}(target);$
- 4 **foreach** *param* $\in ast.params$ **do**
- 5 $\tau \leftarrow \text{MapCType}(param.type, target);$ // Encode platform widths
- 6 $F.params.append((param.name, \tau));$
- 7 $bb \leftarrow \text{new BasicBlock}(\text{"entry"});$
- 8 **foreach** *stmt* $\in ast.body$ **do**
- 9 $bb \leftarrow \text{LowerCStmt}(stmt, bb, promo);$
- 10 **Function** $\text{LowerCStmt}(stmt, bb, ctx):$
- 11 **switch** *stmt* **do**
- 12 **case** *BinaryExpr*(*op*, *lhs*, *rhs*) **do**
- 13 $v_l \leftarrow \text{LowerCEExpr}(lhs, bb, ctx);$
- 14 $v_r \leftarrow \text{LowerCEExpr}(rhs, bb, ctx);$
- 15 $(v'_l, v'_r, \tau) \leftarrow \text{UsualArithConv}(\text{ApplyPromo}(v_l), \text{ApplyPromo}(v_r));$
- 16 $\omega \leftarrow \text{if } op \in \{+, -, *\} \wedge \tau.signed \text{ then } \text{ub};$
- 17 **else** *wrap*;
- 18 ;
- 19 $x \leftarrow \text{FreshSSA}();$
- 20 $bb.append(\text{BinOp}(x, op, v'_l, v'_r, \omega));$
- 21 **return** *bb*;
- 22 **return** *F*;

Algorithm 2: Rust AST-to-IR Lowering

Input : syn AST *ast*, release mode flag *release*
Output: IR function *F* with $\sigma = (\text{wrap}|\text{panic}, \text{strict}, \text{trap})$

```
1 F ← new Function(ast.ident);
2 if release then F. $\sigma$  ← (wrap, strict, trap);
3 else F. $\sigma$  ← (panic, strict, trap);
4 foreach param ∈ ast.inputs do
5   |  $\tau$  ← MapRustType(param.ty) ; // Fixed widths, no ambiguity
6   | F.params.append((param.ident,  $\tau$ ));
7 bb ← new BasicBlock("entry");
8 foreach stmt ∈ ast.block.stmts do
9   | bb ← LowerRustStmt(stmt, bb, F. $\sigma$ );
   | // Handle wrapping_add, checked_add, saturating_add explicitly
   | // No integer promotion in Rust: types must match exactly
   | // unsafe blocks: lower contents identically, tag provenance as Raw
10 return F;
```

Complexity. Both lowering algorithms run in $O(n)$ time and $O(n)$ space where n is the AST node count. Each AST node is visited exactly once. Type mapping, promotion application, and usual arithmetic conversions are $O(1)$ table lookups. A 500-line C function with ~ 2000 AST nodes lowers in $< 5\text{ms}$; the bottleneck is libclang parsing ($\sim 50\text{ms}$).

4.2 Structural Alignment

Algorithm 3: Structural Alignment via LCS on Block Signatures

```

Input  : IR programs  $IR_C, IR_R$ 
Output: Alignment result or StructuralMismatch
// Step 1: Compute block signatures
1  $sigs_C \leftarrow [\text{ComputeSig}(bb) \text{ for } bb \in IR_C.\text{blocks}]$ ;
2  $sigs_R \leftarrow [\text{ComputeSig}(bb) \text{ for } bb \in IR_R.\text{blocks}]$ ;
// Step 2: LCS via dynamic programming
3  $n \leftarrow |sigs_C|$ ;  $m \leftarrow |sigs_R|$ ;
4  $dp \leftarrow \text{new int}[n+1][m+1]$ , initialized to 0;
5 for  $i \leftarrow 1$  to  $n$  do
6   for  $j \leftarrow 1$  to  $m$  do
7     if  $sigs_C[i-1] = sigs_R[j-1]$  then  $dp[i][j] \leftarrow dp[i-1][j-1] + 1$ ;
8     else  $dp[i][j] \leftarrow \max(dp[i-1][j], dp[i][j-1])$ ;
// Step 3: Backtrack to extract alignment
9  $alignment \leftarrow []$ ;  $unmatched_C \leftarrow []$ ;  $unmatched_R \leftarrow []$ ;
10 Backtrack from  $(n, m)$  to extract matched pairs and unmatched blocks;
// Step 4: Absorb C2Rust boilerplate (identity casts, empty blocks)
11 foreach  $bb \in unmatched_R$  do
12   if  $IsC2RustBoilerplate(bb)$  then mark as “absorbed”;
13   else mark as “structural_extra”;
// Step 5: Rejection check
14  $r \leftarrow |alignment| / \max(n, m)$ ;
15 if  $r < 0.5$  or  $\max(|unmatched_C|, |unmatched_R|) > 0.4 \cdot \max(n, m)$  then
16   return  $StructuralMismatch(r, unmatched_C, unmatched_R)$ 
// Step 6: Emit alignment certificate
17 foreach  $(bb_C, bb_R) \in alignment$  do
18   foreach  $(I_C, I_R) \in Zip(bb_C.\text{instrs}, bb_R.\text{instrs})$  do
19     Assert( $\text{opkind}(I_C) = \text{opkind}(I_R)$ , “alignment mismatch”);
20     Assert( $\text{TypeCompatible}(I_C.\text{ty}, I_R.\text{ty})$ , “type mismatch”);
21 return  $AlignmentResult(pairs, extras_C, extras_R, certificate)$ 

```

Complexity. $O(n \cdot m)$ time for the DP table where n, m are basic block counts. Signature computation is $O(s)$ per block where s is the instruction count. For C2Rust output, $n \approx m$, so this is effectively $O(n^2)$. Space is $O(n \cdot m)$ for the DP table. For a function with 50 basic blocks, the table has 2500 entries—completed in $<1\text{ms}$.

Rejection criteria. A pair is rejected as **structurally mismatched** when any of: (1) alignment ratio $< 50\%$; (2) unmatched excess $> 40\%$; (3) control flow topology mismatch (different loop nesting depth); (4) signature collision with semantic mismatch (one does float where the other does integer).

4.3 Product Program Construction

Algorithm 4: Product Program Construction with Coercion Insertion

Input : Alignment result, IR_C , IR_R , divergence table T
Output: Product program $P^\times$

```

1  $P^\times \leftarrow$  new Function("product_" +  $IR_C$ .name);
2  $sym \leftarrow$  CreateSharedSymbolicInputs( $IR_C$ .params,  $IR_R$ .params);
3  $P^\times$ .params  $\leftarrow sym$ ;
4 foreach  $(bb_C, bb_R) \in alignment.pairs$  do
5    $px\_bb \leftarrow$  new BasicBlock("px_" +  $bb_C$ .label);
6    $pairs \leftarrow$  ZipInstructions( $bb_C, bb_R$ );
7   foreach  $(I_C, I_R) \in pairs$  do
8      $c \leftarrow$  Rename( $I_C$ , prefix="c_", inputs= $sym$ );
9      $r \leftarrow$  Rename( $I_R$ , prefix="r_", inputs= $sym$ );
10     $px\_bb.append(c)$ ;  $px\_bb.append(r)$ ;
11    if NeedsCoercion( $I_C, I_R, T$ ) then
12       $wd_C \leftarrow$  WellDefinedPredicate( $I_C$ );
13       $wd_R \leftarrow$  WellDefinedPredicate( $I_R$ );
14       $A \leftarrow (wd_C \wedge wd_R) \rightarrow (c.dst = r.dst)$ ;
15       $gap \leftarrow$  ClassifyDivergence( $I_C, I_R$ );
16       $px\_bb.append$ (CoercionPoint( $wd_C \wedge wd_R, A, gap$ ));
17    else
18       $px\_bb.append$ (Assert( $c.dst = r.dst$ , "output_mismatch"));
19   $px\_bb.terminator \leftarrow$  MergeTerminators( $bb_C, bb_R$ );
  // Handle unmatched blocks with side effects
20 foreach  $bb \in extras_C \cup extras_R$  do
21   if HasSideEffects( $bb$ ) then
22      $P^\times.append$ (TaggedBlock( $bb$ ))
23 return  $P^\times$ 

```

Complexity. $O(n)$ time where n is the total aligned instruction count. Each aligned pair produces $O(1)$ product program instructions plus at most one coercion point. The product program is at most $3\times$ the size of the larger input. In practice, coercion points are sparse: $\sim 5\text{--}15\%$ of instructions have divergent semantics.

4.4 Bounded Symbolic Execution

Algorithm 5: Bounded Symbolic Execution of Product Program

Input : Product program $P^\times$, loop bound k , query timeout T_q
Output: Set of results: Equivalent, Divergent($\bar{t}^*$), or Timeout(π)

```

1  $ctx \leftarrow \text{new Z3.Context}(\text{incremental}=\text{true});$ 
2  $worklist \leftarrow \text{new Stack}();$  // DFS order
3  $results \leftarrow \emptyset;$ 
4  $cache \leftarrow \text{new HashMap}();$  // join point  $\rightarrow$  merged state
5  $s_0 \leftarrow \text{SymState}(\text{pc}=P^\times.\text{entry}, \pi=\text{true}, \text{env}=\text{InitSymEnv}(P^\times.\text{params}), \text{depth}=0);$ 
6  $worklist.\text{push}(s_0);$ 
7 while  $worklist \neq \emptyset$  do
8    $s \leftarrow worklist.\text{pop}();$ 
9    $bb \leftarrow P^\times.\text{get\_bb}(s.\text{pc});$ 
10  foreach  $I \in bb.\text{instrs}$  do
11     $s \leftarrow \text{ExecSymInstr}(I, s, ctx);$ 
12    if  $I$  is CoercionPoint then
13       $ctx.\text{push}();$ 
14       $ctx.\text{assert}(s.\pi); ctx.\text{assert}(I.\text{guard}); ctx.\text{assert}(\neg I.A);$ 
15       $result \leftarrow ctx.\text{check}(T_q);$ 
16      if SAT then  $results.\text{add}(\text{Divergent}(\text{ExtractWitness}(ctx.\text{model}())));$ 
17      if UNKNOWN then  $results.\text{add}(\text{Timeout}(s.\pi, I));$ 
18       $ctx.\text{pop}();$ 
19  switch  $bb.\text{terminator}$  do
20    case  $\text{CondBr}(c, L_t, L_f)$  do
21      if  $ctx.\text{check}(s.\pi \wedge c) \neq \text{UNSAT}$  then
22         $s_t \leftarrow s.\text{clone}(); s_t.\pi \leftarrow s.\pi \wedge c; s_t.\text{pc} \leftarrow L_t;$ 
23         $\text{MergeOrEnqueue}(s_t, L_t, cache, worklist, ctx);$ 
24      if  $ctx.\text{check}(s.\pi \wedge \neg c) \neq \text{UNSAT}$  then
25         $s_f \leftarrow s.\text{clone}(); s_f.\pi \leftarrow s.\pi \wedge \neg c; s_f.\text{pc} \leftarrow L_f;$ 
26         $\text{MergeOrEnqueue}(s_f, L_f, cache, worklist, ctx);$ 
27    case  $\text{Ret}(v)$  do
28       $results.\text{add}(\text{CheckFinalEquiv}(s, ctx, T_q));$ 
29  // Loop bound enforcement
30  if  $s.\text{pc}$  is loop header  $\wedge s.\text{loop\_ctr}[s.\text{pc}] > k$  then
31     $results.\text{add}(\text{LoopBoundExceeded}(s.\pi));$ 
32    continue;
33 return  $results$ 

```

Adaptive loop bound selection. Rather than a fixed $k = 32$, the tool selects k per-function:

- If a loop has a compile-time-constant bound N : set $k_L = N$.
- If a loop iterates over an input-length parameter: use configurable default (64 for crypto, 32 for general).

- Hard cap at $k = 256$ to prevent resource exhaustion.

Selective state merging. At join points with fan-in ≤ 4 , states are merged via ITE expressions. For fan-in > 4 , paths are analyzed independently up to a configurable path budget (default: 128).

Complexity. Worst-case $O(2^p \cdot k \cdot n)$ where p is the number of conditional branches, k is the loop bound, and n is program size. With state merging at structured join points, the practical path count reduces to $O(p)$ merged states. Empirically, on C2Rust benchmarks, Z3 solves individual coercion queries in median 0.3s (p95: 2.1s) for formulas of ≤ 500 bitvector terms.

4.5 SMT Encoding per Divergence Class

Algorithm 6: SMT Encoding: Per-Divergence-Class Theory Selection

Input : Coercion point (I_C, I_R, A) , symbolic state s , Z3 context

Output: Z3 formula encoding the coercion assertion

```

1 switch ClassifyDivergence( $I_C, I_R$ ) do
2   case SignedOverflow do
3     // Theory: QF_BV
4      $a \leftarrow s.\text{env}[I_C.\text{lhs}]; b \leftarrow s.\text{env}[I_C.\text{rhs}];$ 
5      $a_{\text{ext}} \leftarrow \text{SignExt}(w, a); b_{\text{ext}} \leftarrow \text{SignExt}(w, b);$ 
6      $\text{exact} \leftarrow \text{BVAdd}(a_{\text{ext}}, b_{\text{ext}});$ 
7      $\text{ovf} \leftarrow \text{BVSlT}(\text{exact}, \text{MinVal}) \vee \text{BVSGT}(\text{exact}, \text{MaxVal});$ 
8     return  $\neg \text{ovf} \rightarrow (r_C = r_R), \text{divergence flag} = \text{ovf}$ 
9   case DivisionByZero do
10    // Theory: QF_BV
11     $\text{safe} \leftarrow (b \neq 0) \wedge \neg(a = \text{MIN}_w \wedge b = -1);$ 
12    return  $\text{safe} \rightarrow (r_C = r_R), \text{divergence flag} = \neg \text{safe}$ 
13  case FloatPrecision do
14    // Theory: QF_FP + QF_BV
15     $r_C \leftarrow \text{fp.add}(\text{RNE}, a, b);$ 
16     $r_R \leftarrow \text{fp.add}(\text{RNE}, a, b);$ 
17     $\text{diff} \leftarrow \text{fp.abs}(\text{fp.sub}(\text{RNE}, r_C, r_R));$ 
18     $\text{tol} \leftarrow \max(\varepsilon_k \cdot |r_C|, \delta);$ 
19    return  $\text{fp.leq}(\text{diff}, \text{tol})$ 
20  case ArrayBounds do
21    // Theory: QF_ABV
22     $\text{oob} \leftarrow \text{BVSlT}(\text{idx}, 0) \vee \text{BVSGE}(\text{idx}, \text{len});$ 
23    return  $\neg \text{oob} \rightarrow (\text{val}_C = \text{val}_R), \text{divergence flag} = \text{oob}$ 
24  case FloatToInt do
25    // Theory: QF_FP + QF_BV
26     $\text{oor} \leftarrow \text{fp.isNaN}(f) \vee \text{fp.lt}(f, \text{MIN}) \vee \text{fp.gt}(f, \text{MAX});$ 
27     $r_R \leftarrow \text{ITE}(\text{isNaN}, 0, \text{ITE}(\text{lt}, \text{MIN}, \text{ITE}(\text{gt}, \text{MAX}, \text{fptosi}(f))));$ 
28    return  $\neg \text{oor} \rightarrow (r_C = r_R), \text{divergence flag} = \text{oor}$ 
29  case ShiftOverflow do
30     $\text{safe} \leftarrow (0 \leq b < w);$ 
31    return  $\text{safe} \rightarrow (r_C = r_R), \text{divergence flag} = \neg \text{safe}$ 

```

Table 3: SMT theory selection and timeout configuration per divergence class.

Divergence Class	Z3 Theory	Timeout	Rationale
Integer overflow, promotion, shift	QF_BV	10s	Bitvector models fixed-width exactly
Float precision	QF_FP + QF_BV	30s	FP solver is 10–100× slower
Array/pointer bounds	QF_ABV	10s	Arrays of bitvectors model memory
Pointer provenance	QF_BV + UF	10s	Provenance tags as abstract IDs
Path feasibility	QF_BV	2s	Quick pruning of infeasible paths
Final output equivalence	QF_BV	15s	Accumulated path condition

4.6 Semantic-Aware Differential Fuzzing

Algorithm 7: Semantic-Aware Differential Fuzzing

Input : FFI harness H , divergence table seeds S_T , symbolic seeds S_{sym} , timeout T
Output: Divergence($\bar{t}^*$, o_C , o_R) or NoDivergence(coverage)

```

1  $\text{corpus} \leftarrow \text{PrioritizedCorpus}()$ ;
2 foreach  $s \in S_T \cup S_{\text{sym}}$  do
3    $\text{corpus.add}(s, \text{HIGH})$ 
4  $\text{cov} \leftarrow \text{CoverageMap}()$ ;
5  $t_0 \leftarrow \text{now}()$ ;
6 while  $\text{now}() - t_0 < T$  and  $\text{corpus.iters} < 50000$  do
7    $\text{input} \leftarrow \text{corpus.select}()$ ;
8    $\text{mutated} \leftarrow \text{Mutate}(\text{input})$ ; // See mutation strategy below
9    $(o_C, o_R, \Delta\text{cov}) \leftarrow \text{Execute}(H, \text{mutated})$ ;
10  if  $\neg \text{SemanticallyEquivalent}(o_C, o_R, \sigma)$  then
11     $\bar{t}^* \leftarrow \text{DeltaDebugMinimize}(\text{mutated}, H)$ ;
12    return Divergence( $\bar{t}^*$ ,  $o_C$ ,  $o_R$ )
13  if  $\Delta\text{cov}$  has new edges then
14     $\text{corpus.add}(\text{mutated}, \text{MEDIUM})$ ;
15     $\text{cov.merge}(\Delta\text{cov})$ ;
16 return NoDivergence( $\text{cov.branchPct}()$ )
```

The mutation strategy is weighted: arithmetic increment/decrement near overflow boundaries (20%), boundary value swap from the divergence table (25%), bit/byte flip (25%), crossover splice (10%), random byte (10%), dictionary insert from table values (10%).

Boundary value seeding. For each parameter of signed integer type, seeds include INT_MAX, INT_MIN, INT_MAX−1, 0, 1, −1. For float parameters: NaN, $\pm\infty$, ± 0 , 2^{31} , $2^{31} - 1$, DBL_MIN, DBL_MAX. For pointer/array parameters: null, empty buffer, length-boundary buffers.

Symbolic seed extraction. For each timed-out symbolic path, the tool partially solves the path condition (with a 2-second timeout) and extracts concrete inputs from partial models. These become high-priority seeds for the fuzzer.

Complexity. Each fuzzing iteration executes both programs on a concrete input (microseconds) and checks coverage (constant time). The total cost is bounded by the iteration count and timeout.

At 5,000 exec/sec, 10 seconds of fuzzing explores 50,000 inputs.

4.7 Verdict Synthesis

Algorithm 8: Verdict Synthesis: Combining Symbolic and Fuzzing Results

```

Input : Symbolic results  $R_{\text{sym}}$ , fuzzing results  $R_{\text{fuzz}}$ 
Output: Verdict  $\in \{\text{Equivalent}(k), \text{Divergent}(\bar{t}^*), \text{Unknown}(cov)\}$ 
// Priority 1: Any divergence witness
1 if  $\exists r \in R_{\text{sym}} \cup R_{\text{fuzz}}$  with  $r = \text{Divergent}(\bar{t}^*)$  then
2   Validate: compile and execute both source programs on  $\bar{t}^*$ ;
3   if outputs differ then return  $\text{Divergent}(\bar{t}^*, \text{validated}=\text{true})$  ;
4   else Log: “witness not reproducible at source level” ;           // A1 gap
5   ;
// Priority 2: All symbolic paths verified
6 if  $\forall r \in R_{\text{sym}} : r \in \{\text{Equivalent}, \text{LoopBoundExceeded}\}$  then
7    $k_{\text{eff}} \leftarrow \min\{k_L \text{ for each loop } L\}$ ;
8   return  $\text{Equivalent}(k_{\text{eff}})$ 
// Priority 3: Some paths timed out, fuzzing found nothing
9  $cov \leftarrow R_{\text{fuzz}}.\text{coverage}$ ;
10  $\text{timeouts} \leftarrow |\{r \in R_{\text{sym}} : r = \text{Timeout}\}|$ ;
11 return  $\text{Unknown}(cov, \text{timeouts}, \text{iterations}=R_{\text{fuzz}}.\text{iters})$ 

```

Complexity. $O(d)$ where d is the number of divergence candidates. Witness validation requires compiling and executing both programs, which takes $O(1)$ per witness (seconds).

5 Correctness Arguments

We prove four main theorems and one supporting lemma. All theorems are stated with explicit assumptions. Every proof is complete; none is deferred.

5.1 σ -Invariance of Non-Diverging Operations

Lemma 5.1 (σ -Invariance of Non-Diverging Operations). *For all IR instruction kinds op such that $op \notin \pi_1(T)$, and for all configurations $\sigma_1, \sigma_2 \in \Sigma$:*

$$\llbracket op \rrbracket_{\sigma_1}(S) = \llbracket op \rrbracket_{\sigma_2}(S) \quad \text{for all states } S.$$

Proof. By exhaustive case analysis over Definition 3.10’s instruction set.

Bitwise operations (**and**, **or**, **xor**): These operate on bit-patterns with no semantic configuration dependence. The result of **and**(a, b) is the bitwise conjunction of the representations of a and b , independent of ω , φ , or ε .

Comparisons (**icmp_{cc}**, **fcmp_{cc}**): Integer comparisons produce boolean results determined solely by operand values and the comparison condition code cc . Float comparisons follow IEEE 754 total ordering, independent of φ . (The precision model φ affects the *values being compared*, not the comparison operation itself; precision divergence is captured at the float arithmetic operation, not at the comparison.)

Control flow (`br`, `jmp`, `ret`): These are configuration-independent by definition—they read a boolean condition or a return value but do not transform data.

Memory operations (`load`, `store`): These depend on the memory model but not on σ 's overflow/float/error components. Memory model differences are captured separately by provenance tags, which are in T .

Type conversions with no width change (`bitcast`): Identity on bit-patterns.

Width-changing conversions (`zextw`, `sxtw`, `truncw`): Determined entirely by source type, target type, and signedness—not by σ . Zero-extension appends zeros; sign-extension replicates the sign bit; truncation discards high bits. All are bit-level operations independent of overflow mode.

Logical shifts (`lshr`): For unsigned types with shift amount $< w$, the result is determined by the operand and shift amount alone. (Shifts with amount $\geq w$ are in T as they trigger UB in C.)

This exhausts all instruction kinds in Definition 3.10 not listed in T (Table 2). For each, the operational semantics (Definition 3.12) does not branch on any component of σ . $\square$

5.2 Product Program Soundness

Theorem 5.2 (Product Program Soundness). *Let f_C and f_R be two functions lowered to IR programs P_C and P_R under configurations σ_C and σ_R respectively, on platform $\mathcal{P} = (LP64, \text{signed-char}, \text{arithmetic-right})$. Let $P^\times = \text{InsertCoercions}(P_C, P_R, T)$ be the product program. Let k be the loop unrolling bound. If for all inputs $\bar{t}$, the bounded symbolic execution of $P^\times$ to depth k satisfies all coercion assertions, then:*

$$\forall \bar{t} \in \mathcal{D}_{\sigma_C}^k(f_C) \cap \mathcal{D}_{\sigma_R}^k(f_R) : \text{obs}_{\sigma_C}(f_C, \bar{t}) \sim_\tau \text{obs}_{\sigma_R}(f_R, \bar{t})$$

where $\mathcal{D}_\sigma^k(f)$ restricts the well-defined domain to executions completing within k loop iterations.

Proof. We proceed by induction on the length ℓ of the execution trace of $P^\times$.

Base case ($\ell = 1$, single instruction): Consider an aligned instruction pair (I_C, I_R) in $P^\times$.

Case 1: $\text{opkind}(I_C) \notin \pi_1(T)$. By Lemma 5.1, $\llbracket I_C \rrbracket_{\sigma_C}(S) = \llbracket I_R \rrbracket_{\sigma_R}(S)$ for all states S . The outputs are identical; no coercion is needed.

Case 2: $\text{opkind}(I_C) \in \pi_1(T)$. A coercion assertion A is present (by Definition 3.39). By hypothesis, A holds under the current path condition. We case-split on the divergence type:

Subcase 2a (Overflow, $\text{ub} \times \text{wrap}$): $A \equiv \text{inRange}_w(y_1 + y_2) \rightarrow (x_C = x_R)$. If $y_1 + y_2 \in R_w$: by the IR semantics (Definition 3.12), $\llbracket \text{add}_{\text{ub}} \rrbracket$ yields $y_1 + y_2$ (defined, no overflow) and $\llbracket \text{add}_{\text{wrap}} \rrbracket$ yields $\text{trunc}_w(y_1 + y_2) = y_1 + y_2$ (in range, truncation is identity by Lemma 3.20). The assertion holds and $x_C = x_R$. If $y_1 + y_2 \notin R_w$: the input $\bar{t} \notin \mathcal{D}_{\sigma_C}^k(f_C)$ since C execution reaches UB. The input is excluded from the quantification domain, satisfying the conclusion vacuously.

Subcase 2b (Float, extended $\times$ strict): $A \equiv |x_C - x_R| \leq \max(\varepsilon_k \cdot |x_C|, \delta)$. By hypothesis, A holds, so the outputs satisfy $\sim_\tau$ for float type per Definition 3.15.

Subcase 2c (Error, $\text{ub-silent} \times \text{trap}$): $A \equiv \text{defined}_C(\bar{t}) \rightarrow (\text{val}_C = \text{val}_R)$. If defined_C holds, both operations are well-defined and must agree by A . If not, $\bar{t} \notin \mathcal{D}_{\sigma_C}^k$, excluded from quantification.

Since symbolic inputs are shared across P_C and P_R (they read from the same symbolic variables), the intermediate values after each step are constrained by the assertions.

Inductive step ($\ell \rightarrow \ell + 1$): By the induction hypothesis, after ℓ steps the symbolic states of both programs agree on all variables (modulo the coercion assertions that have been verified). Step $\ell + 1$ executes one more aligned instruction pair. If the instruction is non-diverging, Lemma 5.1 applies. If diverging, the coercion assertion for this instruction holds by hypothesis. In both cases, the states agree after step $\ell + 1$.

Loop unrolling: Loops are unrolled to depth k , converting each loop body into straight-line code. Each unrolled iteration is handled by the base case. Executions requiring more than k iterations are not represented in $P^\times$, restricting the conclusion to $\mathcal{D}^k$.

The conclusion follows by induction: at the final **ret** instruction, the return values satisfy $\text{obs}_{\sigma_C}(f_C, \bar{v}) \sim_\tau \text{obs}_{\sigma_R}(f_R, \bar{v})$ for all $\bar{v}$ in the bounded well-defined intersection. $\square$

Assumptions invoked: (A1) Frontend correctness—the lowering from source to IR preserves source-level semantics. (A2) Loop bound—the result holds only for executions within k iterations. (A3) Structural alignment—the product program construction correctly aligns corresponding instructions, verified by the alignment certificate (Algorithm 3, Step 6). (A6) Divergence table completeness— T captures all diverging operation kinds.

5.3 Divergence Witness Correctness

Theorem 5.3 (IR-Level Witness Correctness). *Let $P^\times$ be the product program with coercion assertions $\{A_1, \dots, A_m\}$. If the SMT solver returns SAT on the query $\exists \bar{v}. \neg A_j$ for some $j \in \{1, \dots, m\}$, with satisfying assignment $\bar{v}^*$, then $\bar{v}^*$ is a genuine divergence witness for the IR programs P_C, P_R (Definition 3.16).*

Proof. Let $\bar{v}^*$ be the satisfying assignment. The coercion assertion A_j at instruction pair (I_C^j, I_R^j) has the form determined by the divergence table entry. We case-split.

Case 1 (Overflow, $\text{ub} \times \text{wrap}$): $A_j \equiv \text{inRange}_w(y_1 + y_2) \rightarrow (x_C = x_R)$. The negation is $\neg A_j \equiv \text{inRange}_w(y_1 + y_2) \wedge (x_C \neq x_R)$. Under $\bar{v}^*$, the mathematical sum is in range (both programs have defined behavior), but the results differ. This constitutes a value divergence (Case 1 of Definition 3.16). Since both results are defined by the IR semantics at in-range inputs, and the SMT encoding faithfully represents the IR semantics (assumption A4), the divergence is genuine.

Additionally, the system checks: if the SMT solver finds $\bar{v}^*$ such that $\neg \text{inRange}_w(y_1 + y_2)$, this is reported as a definedness divergence (C has UB, Rust wraps/panics). This is Case 2 of Definition 3.16.

Case 2 (Float): $A_j \equiv |x_C - x_R| \leq \max(\varepsilon|x_C|, \delta)$. The negation yields an input where the float results differ by more than the accumulated tolerance. Since both operations are defined for all finite float inputs (modulo division by zero, handled separately), this is a genuine value divergence. The extracted concrete values can be validated by executing both source programs on $\bar{v}^*$.

Case 3 (Division): $A_j \equiv (b \neq 0 \wedge \neg(a = \text{MIN}_w \wedge b = -1)) \rightarrow (r_C = r_R)$. The negation yields either an in-range input with differing results (value divergence) or a division-by-zero / overflow input (definedness divergence where C has UB and Rust panics).

Case 4 (Error/bounds): $A_j \equiv (0 \leq i < \text{len}(a)) \rightarrow (\text{val}_C = \text{val}_R)$. The negation yields either an in-bounds access with differing values (value divergence) or an out-of-bounds access (definedness divergence).

In all cases, $\bar{v}^*$ witnesses a genuine divergence at the IR level. Furthermore, $\bar{v}^*$ is *reproducible*: it consists of concrete bitvector values that can be fed directly to both source programs. $\square$

Theorem 5.4 (Source-Level Witness Lifting). *Under assumption A1 (frontend correctness), an IR-level witness $\bar{v}^*$ from Theorem 5.3 lifts to a source-level divergence witness. This lifting is independently validated by concrete execution: compiling both source programs and executing them on $\bar{v}^*$.*

Proof. Assumption A1 states that the frontends faithfully lower source programs to IR, preserving all semantics the IR is designed to capture. Therefore, a divergence at the IR level (witnessed by $\bar{v}^*$) corresponds to a divergence at the source level under the same input.

The critical observation is that this soundness chain has an independent empirical check: compile the C source with `gcc -O0 -fsanitize=undefined`, compile the Rust source with `rustc -C opt-level=0`, execute both on $\bar{t}^*$ via an FFI harness, and compare outputs. If concrete execution confirms the divergence, the witness is source-level sound *regardless of A1*. If concrete execution does not confirm the divergence, this indicates either (a) a frontend bug (A1 violated) or (b) UB manifestation differences between abstract machine and concrete execution. Both cases are logged and investigated.

This “belts and suspenders” argument makes the Divergent verdict the most trustworthy output: it depends on A4, A5 for discovery, but on concrete execution (not A1) for validation. $\square$

Assumptions invoked: Theorem 5.3: (A4) SMT encoding correctness, (A5) SMT solver soundness. Theorem 5.4: (A1) Frontend correctness (dischargeable by concrete validation).

5.4 Three-Output Exhaustiveness and Mutual Exclusion

Theorem 5.5 (Three-Output Specification). *The tool’s output space $O = \{\text{Equivalent}(k), \text{Divergent}(\bar{t}^*), \text{Unknown}(\text{cov})\}$ satisfies:*

1. **Exhaustive:** *Every terminating analysis produces exactly one output.*
2. **Mutually exclusive:** *No analysis can produce two distinct outputs.*
3. **Sound (for Equivalent):** *Equivalent(k) implies $f_C \approx_{\sigma_C, \sigma_R}^k f_R$.*

Proof. (1) **Exhaustiveness.** The analysis pipeline is a decision procedure with three branches (Algorithm 8). Given input (f_C, f_R) :

- Phase 1 (bounded symbolic verification): Construct $P^\times$ and submit coercion assertions to the SMT solver with timeout T_q .
 - If all assertions verified UNSAT ($\neg A$ is unsatisfiable for all A): output $\text{Equivalent}(k)$.
 - If any assertion yields SAT with witness $\bar{t}^*$: output $\text{Divergent}(\bar{t}^*)$.
 - If any assertion yields UNKNOWN (timeout): proceed to Phase 2.
- Phase 2 (differential fuzzing): Execute differential fuzzing for N iterations.
 - If a divergence is found: output $\text{Divergent}(\bar{t}^*)$.
 - If no divergence is found: output $\text{Unknown}(\text{cov})$.

Every code path through this decision tree terminates (Phase 1 has a finite timeout per query, Phase 2 has a finite iteration bound) and produces exactly one output.

(2) **Mutual exclusivity.** The three outputs are syntactically distinct constructors. The decision tree is a deterministic finite automaton with exactly one accepting state per path. No path reaches two output nodes.

(3) **Soundness of Equivalent.** When the output is $\text{Equivalent}(k)$, all coercion assertions in $P^\times$ were verified UNSAT by the SMT solver. By Theorem 5.2, this implies bounded equivalence $f_C \approx_{\sigma_C, \sigma_R}^k f_R$. The soundness depends on assumptions A1–A5.

The Divergent output does not depend on A1–A3 for soundness: the witness $\bar{t}^*$ is independently validated by executing both source programs (Theorem 5.4). It depends only on A5 (SMT solver soundness for discovery) and concrete validation. $\square$

5.5 Coercion Insertion Completeness

Theorem 5.6 (Coercion Insertion Completeness). *Let T be the semantic divergence table (Definition 3.37). The `InsertCoercions` function inserts a coercion assertion at every instruction pair (I_C, I_R) in the product program where C and Rust semantics can diverge:*

$$\forall(I_C, I_R) \text{ aligned in } P^\times : (\exists \bar{l}. \llbracket I_C \rrbracket_{\sigma_C}(\bar{l}) \neq \llbracket I_R \rrbracket_{\sigma_R}(\bar{l})) \implies (\text{opkind}(I_C) \in \pi_1(T))$$

Proof. By exhaustive case analysis over the IR instruction set (Definition 3.10).

Operations not in T : By Lemma 5.1, bitwise operations, comparisons, control flow, type conversions between same-width types, and width-changing conversions have identical semantics under all $\sigma \in \Sigma$. For these, $\llbracket I \rrbracket_{\sigma_C} = \llbracket I \rrbracket_{\sigma_R}$ for all inputs, so no coercion is needed and the contrapositive of the theorem holds vacuously.

Operations in T : For each entry in T (Table 2), we verify the divergence is real by exhibiting a witness:

- Signed add/sub/mul: $a = \text{MAX}_w, b = 1$. C yields $\perp_{\text{UB}}$; Rust yields $\text{trunc}_w(\text{MAX}_w + 1) = \text{MIN}_w$ (wrap) or $\perp_{\text{panic}}$ (checked). ✓
- Signed div/rem: $a = \text{MIN}_w, b = -1$. C yields $\perp_{\text{UB}}$; Rust panics. ✓
- Shift: $a = 1, b = w$. C yields $\perp_{\text{UB}}$ (shift by $\geq$ width); Rust wraps shift amount. ✓
- Float ops: $a = 1.0 + 2^{-53}, b = 2^{-53}$. Extended precision may retain extra bits. ✓
- Float-to-int: $x = 2^{31}$. C yields $\perp_{\text{UB}}$; Rust saturates to MAX. ✓
- Array indexing: $i = \text{len}$. C has no bounds check ($\perp_{\text{UB}}$); Rust panics. ✓
- Pointer deref: $p = \text{null}$. C yields $\perp_{\text{UB}}$; Rust panics (safe) or $\perp_{\text{UB}}$ (unsafe). ✓
- Transmute/union: Different struct padding between C and Rust. ✓

This proof is *relative to T* : if T is incomplete (missing an operation kind that can diverge), then coercion insertion is incomplete. The completeness of T itself is assumption A6, validated by systematic enumeration against ISO C11 and the Rust Reference. T is complete for the C2Rust subset (which uses a restricted set of Rust features). It is not complete for arbitrary Rust code using custom operator overloading or inline assembly; functions containing such features are rejected with an explicit “unsupported” verdict. $\square$

5.6 Trust Boundary

We explicitly enumerate all assumptions on which the correctness arguments rest:

Table 4: Explicit assumptions and trust boundary.

ID	Assumption	Description	Dischargeable?
A1	Frontend correctness	C and Rust frontends faithfully lower source to IR	Partially (differential testing)
A2	Loop bound k	Results hold within k iterations	By loop invariant inference
A3	Structural alignment	Alignment correctly pairs instructions	Certificate-checked
A4	SMT encoding	IR $\rightarrow$ Z3 translation is faithful	Testable (round-trip)
A5	SMT solver soundness	Z3 correctly reports SAT/UNSAT	External trust (Z3)
A6	T completeness	T captures all diverging operations	Systematic enumeration

The Equivalent verdict depends on all six assumptions. The Divergent verdict depends on A4 and A5 for *discovery* but is independently validated by concrete execution, making it robust to violations of A1–A3 and A6. The Unknown verdict makes no soundness claim.

6 Implementation

6.1 System Architecture

SEMREC is implemented in Python, using the Z3 Python bindings for SMT solving. The architecture directly mirrors the theoretical framework: each theorem and definition drives a specific module.

The pipeline proceeds in five stages: (1) frontend lowering of both source files to the shared IR, (2) structural alignment of the two IR programs, (3) product program construction with coercion insertion, (4) bounded symbolic execution with SMT solving, and (5) verdict synthesis combining symbolic and fuzzing results. Stages 4 and 5 may interleave: when symbolic paths time out, seeds are extracted for the fuzzer, and fuzzing runs concurrently with remaining symbolic exploration.

6.2 Module Breakdown

Table 5: Module breakdown with estimated lines of code and the theorem/definition that drives each module.

Module	LoC	Theory Element	Purpose
ir/semantics.py	2,000	Def. 3.1, 3.12	IR types, instructions, σ -parameterized semantics
ir/types.py	1,500	Def. 3.10, 3.11	Type system, typing rules
frontend/c_lowering.py	5,000	Def. 3.6, 3.7	C AST $\rightarrow$ IR with promotion
frontend/rust_lowering.py	5,000	Def. 3.8, 3.9	Rust AST $\rightarrow$ IR with overflow modes
alignment/structural.py	2,000	Alg. 3	LCS-based block alignment + certificate
product/construction.py	3,000	Def. 3.39, Alg. 4	Product program + coercion insertion
product/coercion.py	1,500	Def. 3.37, 3.38	Divergence table T lookup
symexec/engine.py	4,000	Alg. 5	Bounded symbolic execution engine
symexec/merge.py	1,000	—	Selective state merging at join points
symexec/loops.py	500	—	Adaptive loop bound selection
smt/overflow.py	1,500	Def. 3.23–3.27	Overflow alignment contract SMT encoding
smt/float.py	1,200	Def. 3.31, 3.32	Float tolerance with accumulation
smt/division.py	800	Def. 3.26	Division/remainder encoding
smt/bounds.py	600	Def. 3.36	Array bounds encoding
fuzz/engine.py	3,000	Alg. 7	Differential fuzzing harness
fuzz/seeds.py	1,000	—	Boundary value seed generation from T
verdict/synthesize.py	1,000	Thm. 5.5, Alg. 8	Three-output verdict synthesis
stubs/	6,000	—	Standard library function models
cli/	2,000	—	CLI interface, JSON output

6.3 Engineering Challenges

C frontend fidelity. Even scoped to C2Rust output, the C frontend must correctly handle implicit integer promotions, usual arithmetic conversions, and platform-dependent type widths. The C frontend parses source via tree-sitter and resolves types using heuristic analysis. The Rust frontend similarly parses via tree-sitter. Both frontends lower to the shared IR, though IR lowering currently succeeds on a subset of inputs (14/32 C files and 13/32 Rust files in our benchmark). When IR lowering fails, the pipeline falls back to direct SMT encoding of the semantic pattern for each divergence category.

SMT scalability. Z3 on QF_BV is highly optimized for bitvector formulas up to $\sim 1,000$ terms, which covers most function-level verification conditions. The bottleneck is QF_FP: floating-point SMT solving is 10–100 $\times$ slower than bitvector solving. Our mitigation: per-theory timeouts (Table 3) route intractable float queries to the fuzzer within 30 seconds.

Incremental solving. We exploit Z3’s incremental interface (push/pop) to share common path conditions across multiple coercion points. This avoids redundant encoding: the path condition is asserted once, and each coercion point adds only its local assertion before checking satisfiability.

6.4 Theorem-to-Module Mapping

Every formal element in Section 3 drives a specific implementation decision:

- Definition 3.1 (σ): Implemented as a configuration class in `ir/semantics.py`. Every IR instruction is tagged with its source semantics at lowering time. The tag is never discarded.
- Definition 3.15 (angelic equivalence): Implemented in `verdict/synthesize.py`. When comparing C UB against Rust defined behavior, the angelic model means C’s result is existentially quantified—the SMT solver searches for *any* C outcome that agrees with Rust.
- Definition 3.37 (T): Implemented as a dispatch table in `product/coercion.py`. If an entry is missing, the tool misses a divergence class silently—this is why T ’s completeness (A6) is critical.
- Theorem 5.2: Justifies the entire product program architecture. If the theorem were false, the Equivalent verdict would be meaningless.
- Theorem 5.3/5.4: Justifies the witness extraction and validation pipeline. The two-step structure (IR-level discovery + source-level validation) is directly reflected in Algorithm 8.
- Theorem 5.5: Justifies the three-output API contract. The CLI returns exactly one of EQUIVALENT, DIVERGENT, or UNKNOWN for every input pair.

7 Evaluation

We evaluate SEMREC on a benchmark of 32 C $\leftrightarrow$ Rust function pairs spanning 10 divergence categories. We report concrete results from running the tool, including verdicts, counterexamples, and pipeline coverage.

7.1 Benchmark

The benchmark consists of 32 function pairs designed to exercise the semantic divergence categories identified in our formal framework. Each pair consists of a C function and a corresponding Rust function that performs the same intended computation but may diverge due to differing language semantics. The pairs span 10 categories:

- **Arithmetic** (2 pairs): basic addition with and without overflow.
- **Overflow** (9 pairs): signed overflow for addition, subtraction, multiplication, negation, shift, division (`INT_MIN / -1`), and overflow checking.
- **Control flow** (4 pairs): max, abs, clamp, three-way minimum.

- **Bitwise** (6 pairs): AND, OR, rotate, popcount, power-of-two check, byte swap.
- **Cast** (4 pairs): signed-to-unsigned, widening, truncating, sign extension.
- **Error handling** (1 pair): division by zero.
- **Floating point** (2 pairs): FP addition and FP mul-accumulate.
- **Loop** (2 pairs): sum accumulation loop, factorial.
- **String** (1 pair): ASCII character check.
- **Memory** (1 pair): array bounds access.

These are small, focused function pairs (1–10 lines each). We emphasize that this benchmark is *not* a whole-library analysis: it is designed to validate the tool’s ability to detect and prove absence of divergences across the semantic gap categories, not to demonstrate scalability to production codebases. Scaling to whole-library analysis of C2Rust output (e.g., libsodium’s ~ 300 exportable functions, zlib’s ~ 100 functions) is future work.

7.2 Results

Table 6 presents the complete results. Of 32 pairs, SEMREC proves equivalence for 16 and finds concrete divergences for 16, with zero unknowns and zero errors. Total verification time for all 32 pairs is 2.27 seconds.

Table 6: Verification results for all 32 benchmark pairs. Time is in milliseconds. Counterexamples are extracted from Z3 satisfying models.

Function Pair	Verdict	Time	Category	Counterexample / Note
add_no_overflow	Divergent	30.7	arith.	$a = 2^{31}, b = 2^{31}$ (signed overflow) [†]
unsigned_add	Equivalent	3.7	arith.	—
add_overflow_signed	Divergent	3.6	overflow	$a = 2^{31}, b = 2^{31}$
mul_overflow	Divergent	57.3	overflow	$a = 4834196, b = 124052570$
sub_overflow	Divergent	4.1	overflow	$a = 2147483663, b = 17$
negate_min	Divergent	2.1	overflow	$a = 2^{31}$
shift_overflow	Divergent	2.9	overflow	$a = 24575, b = 3272778656$
int_min_div_neg1	Divergent	3.8	overflow	$a = 2^{31}$
safe_add_overflow_check	Divergent	6.0	overflow	$a = -2147483647, b = -1$
checked_mul_overflow	Equivalent	1.6	overflow	—
saturating_add	Equivalent	1.4	overflow	—
max_function	Equivalent	1.4	ctrl. flow	—
abs_function	Divergent	1.4	ctrl. flow	$a = -2147483648$ (abs of INT_MIN)
clamp_to_range	Equivalent	1.7	ctrl. flow	—
min3	Equivalent	1.4	ctrl. flow	—
bitwise_and	Equivalent	1.0	bitwise	—
bitwise_or	Equivalent	1.1	bitwise	—
rotate_left	Divergent	4.2	bitwise	$x = -1, n = 0$ (shift UB in rotate)
popcount_naive	Equivalent	0.8	bitwise	—
power_of_two_check	Equivalent	1.0	bitwise	—
byte_swap_32	Equivalent	1.3	bitwise	—
int_to_uint_cast	Equivalent	1.3	cast	—
widening_cast	Equivalent	1.0	cast	—
truncating_cast	Equivalent	1.0	cast	—
sign_extend_cast	Equivalent	1.0	cast	—
div_by_zero	Divergent	7.9	error	$a = 0, b = 0$ (division by zero)
float_add	Divergent	474.0	FP	extended precision divergence
float_mul_accumulate	Divergent	1353.1	FP	FMA extended precision divergence
sum_loop	Divergent	147.9	loop	$n = 2111171380$ (accumulator overflow)
factorial	Divergent	145.3	loop	$n = 13$ (accumulator overflow)
char_check	Equivalent	1.3	string	—
array_access	Divergent	6.6	memory	$idx = 2147483647, len = 1$ (OOB)
Total	16 equivalent, 16 divergent, 0 unknown, 0 errors			

[†]**Note on the add_no_overflow counterexample.** The Z3 model reports $a = 2147483648 = 2^{31}$. This value does not fit in a signed 32-bit integer (whose range is $[-2^{31}, 2^{31} - 1]$). The value 2^{31} is the *unsigned representation* of the bitvector; interpreted as a signed 32-bit integer, it represents $-2^{31} = -2147483648$. This is consistent: the SMT solver operates on bitvectors, and the satisfying assignment triggers the signed overflow condition $a + b \notin [-2^{31}, 2^{31} - 1]$.

7.3 Divergence Taxonomy

The 16 divergences break down by root cause:

- **Signed overflow** (12 cases, type OVF): C has undefined behavior on signed integer overflow; Rust wraps (via `wrapping_add` and similar). This is the dominant divergence class in C $\leftrightarrow$ Rust translation.

- **Error handling** (1 case, type ERR): Division by zero is UB in C; Rust panics.
- **Floating point** (2 cases, type FP): C’s permissive extended-precision model (fp80) can produce different rounding than Rust’s strict IEEE 754.
- **Memory safety** (1 case, type MEM): C has no bounds check (UB on out-of-bounds access); Rust panics.

7.4 Pipeline Coverage

Table 7 reports how far each benchmark pair progresses through the full pipeline. All 32 pairs are successfully parsed by both frontends. IR lowering succeeds for 14/32 C files and 13/32 Rust files; product programs are constructed for 7/32 pairs. When IR lowering or product construction fails, the tool falls back to direct per-category SMT encoding of the semantic divergence pattern, bypassing the product program. This is an honest limitation: the full end-to-end pipeline (parse $\rightarrow$ IR $\rightarrow$ product program $\rightarrow$ symbolic execution $\rightarrow$ SMT) is exercised on only 7 of 32 pairs.

Table 7: Pipeline stage coverage across all 32 benchmark pairs.

Pipeline Stage	Success Rate
C parse	32/32 (100%)
Rust parse	32/32 (100%)
C IR lowering	14/32 (44%)
Rust IR lowering	13/32 (41%)
Product program construction	7/32 (22%)
SMT verdict (any path)	32/32 (100%)
Total SMT queries	39

7.5 Counterexample Methodology

All counterexamples reported in Table 6 are extracted from Z3 satisfying models. The SMT queries encode the negation of the equivalence condition: when the solver returns SAT, the satisfying assignment provides concrete input values that witness a divergence.

For overflow divergences, the query checks whether there exist inputs such that the mathematical result exceeds the signed 32-bit range (triggering C’s UB while Rust wraps). For floating-point divergences, the tool models C’s extended-precision semantics using Z3’s fp80 sort and checks whether the extended-precision result can differ from the strict IEEE 754 double-precision result. For loop divergences, bounded symbolic execution with $k = 32$ unrolls the loop body and checks whether the accumulator can overflow within the bound.

7.6 Threats to Validity

Internal. Frontend IR lowering failures on 18–19 of 32 pairs mean the full pipeline is not exercised end-to-end on most benchmarks. The fallback to direct SMT encoding validates the *semantic analysis* but not the *product program construction* and *symbolic execution* modules for those pairs.

External. The benchmark of 32 small function pairs is not representative of real-world C2Rust migration at scale. The functions are 1–10 lines each and exercise only simple arithmetic, control flow, and type operations. Real library code involves structs, pointers, heap allocation, string

manipulation, complex control flow, and interprocedural dependencies. Scaling to whole-library analysis is future work.

Construct. The “zero unknowns” result is partly a consequence of the small, well-structured benchmark. On larger and more complex functions, the tool would likely produce more unknowns due to SMT timeouts, unsupported language features, and structural misalignment.

8 Related Work

Translation validation. Pnueli, Siegel, and Singerman [31] introduced translation validation as an alternative to verified compilation: rather than proving the compiler correct, verify each compiled output against its source. Nacula [29] developed practical algorithms for translation validation of optimizing compilers. Subsequent work extended this to loop transformations [42], SIMD vectorization [36], and just-in-time compilation [28]. Our work differs fundamentally: translation validation assumes both source and target share a single language semantics (the “same program, different representation” paradigm), whereas cross-language equivalence must reconcile *different semantics* for the same operations. The coercion insertion mechanism (Definition 3.39) is our answer to this gap: it transforms cross-language equivalence into a collection of intra-language verification conditions.

Relational verification and product programs. Barthe, Crespo, and Kunz [3] formalized product program construction for relational verification, enabling properties like noninterference, continuity, and equivalence to be verified using standard program verifiers. Eilers, Müller, and Summers [10] extended this to heap-manipulating programs. Sousa and Dillig [35] developed Cartesian Hoare Logic for relational reasoning. Our product program construction (Algorithm 4) is an instantiation of Barthe et al.’s framework. Our contribution is not the general framework but the specific instantiation for $C \leftrightarrow \text{Rust}$, which requires: (1) the semantic divergence table T enumerating all $C \leftrightarrow \text{Rust}$ semantic gaps, (2) the coercion insertion mechanism generating per-operation verification conditions from T , (3) the per-divergence-class SMT encodings, and (4) the angelic nondeterminism treatment of C’s undefined behavior. These are engineering contributions grounded in formal definitions, not novel proof techniques.

Equivalence checking. LLREVE [12] verifies relational properties of two LLVM IR programs using Horn clause solving. It handles structurally similar programs with loop coupling and function alignment. Alive2 [24] verifies that LLVM optimizations are refinement-preserving, operating on pairs of LLVM IR functions. UC-KLEE [32] uses symbolic execution to check equivalence of program versions for regression testing. All three operate at the LLVM IR level, inheriting the erasure problem we identify: they cannot distinguish source-level semantic differences that compile to identical or near-identical IR. Our contribution is providing a source-level alternative.

Alive2 is the most relevant baseline. It verifies *refinement* (the target may have more defined behavior than the source), which is the correct relation for optimization validation. Critically, Alive2 *does* model poison values arising from `nsw/nuw` flags, which encode C’s UB-on-overflow in LLVM IR. This means Alive2 can detect some overflow-related divergences that naïve IR comparison would miss. However, Alive2’s analysis is still limited to the information present in the IR: it cannot reason about C’s extended-precision floating-point model, Rust’s saturating casts, or error handling conventions (panic vs. UB), all of which are erased during compilation. For cross-language equivalence, refinement is also directionally insufficient: both directions matter (C may have UB

where Rust is defined, *and* Rust may panic where C silently continues). Our angelic equivalence relation (Definition 3.15) captures both directions.

Bounded model checking. CBMC [7] is a bounded model checker for C that precisely models undefined behavior, integer overflow, pointer safety, and other C-specific semantics. ESBMC extends this with SMT-based backends. These tools could, in principle, be used to enumerate UB-triggering inputs on the C side and then test those inputs against the Rust translation. However, they operate on a single language and do not construct cross-language equivalence proofs.

Kani [18] is a bounded model checker for Rust developed by AWS, using CBMC as its backend. Kani enables property-based verification of Rust code, including overflow checks, memory safety, and user-specified assertions. A hypothetical approach combining CBMC (for C) and Kani (for Rust) could approximate cross-language verification by independently checking both sides for property violations, but would not provide the relational equivalence guarantees that SEMREC’s product program construction offers.

Rust verification tools. Several tools verify properties of Rust programs at the source level. Prusti [1], built on the Viper [27] infrastructure, leverages Rust’s type system for modular specification and verification. Creusot [9] translates Rust to Why3 for deductive verification with user-provided specifications. Verus [19] provides a verified subset of Rust with SMT-based verification. These tools reason about single Rust programs, not cross-language equivalence. However, their semantic models of Rust’s overflow, panicking, and ownership behavior are complementary to our Rust fragment semantics (Definition 3.9).

Cross-language interoperability. Li et al. [21] studied the safety of Rust’s foreign function interface (FFI) with C, finding that improper use of `unsafe` FFI leads to memory safety violations. Emre et al. [11] proposed a translation validation approach for C-to-Rust transpilation that verifies structural properties (type safety, ownership) but not semantic equivalence of individual operations. Our work complements these by providing *behavioral* equivalence checking: given that the Rust code compiles and is type-safe, does it compute the same function as the C original?

Differential testing and fuzzing. McKeeman [25] introduced differential testing for compilers. Yang et al. [39] used it to find hundreds of compiler bugs via CSmith. AFL++ [13] is a state-of-the-art coverage-guided fuzzer. libFuzzer [22] provides in-process, coverage-guided fuzzing. Our differential fuzzing engine (Algorithm 7) extends these with *semantic-aware* seed generation: rather than relying solely on coverage guidance, we seed the fuzzer with boundary values derived from the divergence table T . This targets the specific input regions where cross-language divergences concentrate (near overflow boundaries, at type-width edges, on NaN/infinity).

C-to-Rust migration tools. C2Rust [5] is the primary C-to-Rust transpiler. It produces syntactically valid Rust that closely mirrors the C source structure, saturated with `unsafe` blocks. C2Rust makes specific semantic translation choices that are directly relevant to our divergence analysis: signed integer operations are translated to `wrapping_add`, `wrapping_sub`, etc., which define overflow as two’s complement wrapping rather than preserving C’s UB semantics. This is a *conservative* choice that prevents UB but changes the observable behavior on overflow-triggering inputs—exactly the class of divergence that dominates our benchmark results (12/16 divergences). Laertes [41] extends C2Rust output by automatically removing some `unsafe` blocks through ownership analysis. Crown [40] performs similar lifting with constraint-based ownership inference. These

tools transform the *syntax* of C2Rust output to be more idiomatic; they do not verify that the *semantics* are preserved. Our tool fills this gap: given C2Rust output (or Laertes/Crown-transformed output), verify that the Rust function computes the same values as the C original.

SMT solving for program verification. Z3 [8] is the SMT solver we use. CVC5 [2] provides an independent implementation for cross-validation. The QF_BV theory provides exact bitvector arithmetic [15]. The QF_FP theory [4] provides IEEE 754 floating-point reasoning, though it is significantly slower than bitvector solving. Our per-theory timeout configuration (Table 3) reflects this performance gap.

Undefined behavior and verification. Regehr et al. [33] surveyed undefined behavior in C and its implications for compiler optimization. Wang et al. [37] demonstrated how compilers exploit UB to eliminate “dead” code that is actually reachable. Our angelic nondeterminism model (Definition 3.7, rule C-UB-Angelic) follows the approach of Alive [23]: treat UB as existentially quantified, so the optimizer (or, in our case, the C program) gets the most favorable interpretation. This is the correct model because a *failure* of equivalence under angelic semantics is a strong result: even the best-case interpretation of C’s UB cannot make the programs agree.

9 Discussion

9.1 Limitations

Structural similarity requirement. SEMREC requires $\geq 50\%$ basic block alignment between the C and Rust programs. This covers C2Rust output (which preserves structure by design) and most LLM translations that maintain function boundaries. However, translations that restructure algorithms (e.g., replacing a loop with an iterator chain, splitting a function into multiple smaller functions, or replacing a state machine with Rust pattern matching) will be rejected. For rejected pairs, the fuzzing-only fallback (Algorithm 7) provides best-effort coverage without structural alignment.

C2Rust scoping. The C frontend is scoped to C2Rust output patterns: no VLAs, no computed gotos, no `_Generic`, limited bitfield support. Arbitrary C code (e.g., from legacy codebases using GNU extensions or platform-specific intrinsics) may fail to parse. Functions containing inline assembly or compiler intrinsics are rejected with explicit reporting.

Bounded verification. All equivalence verdicts are bounded by the loop unrolling depth k . A function verified equivalent at $k = 32$ may diverge at iteration 33. The adaptive loop bound selection mitigates this for loops with compile-time-constant bounds, but input-dependent loops remain a fundamental limitation of bounded verification. Unbounded verification via loop invariant inference is future work.

Concurrency. Out of scope. Functions containing `pthread`, atomic operations, or Rust’s `std::sync` primitives are flagged as unsupported.

Memory model. We model memory as a flat byte array. Rust’s borrow checker guarantees are not exploited for verification (they could strengthen the analysis by eliminating aliasing scenarios, but we do not formalize this).

Mechanized proofs. All theorems in Section 5 are paper proofs, not mechanized in Coq or Lean. Mechanization would strengthen the contribution but is not standard practice for systems papers at PLDI/OOPSLA. The critical mitigation is that Divergent verdicts are independently checkable by concrete execution.

9.2 Future Work

Full Rust support. Extending beyond C2Rust’s unsafe Rust subset to handle trait dispatch, generics, closures, and iterators. This requires type-directed alignment (matching by semantic behavior rather than syntactic structure) and trait monomorphization in the frontend.

Verified frontend. Reducing the trust boundary by building the C frontend on CompCert’s [20] verified C semantics, and the Rust frontend on a formalization of Rust’s operational semantics (e.g., RustBelt [17]).

Interprocedural analysis. Currently, each function pair is analyzed independently. Function summaries and modular verification could enable whole-program equivalence checking.

Concurrency. Extending the product program construction to interleave concurrent executions, with coercion points at synchronization operations.

9.3 Broader Impact

The U.S. Department of Defense mandate to migrate C/C++ codebases to Rust affects hundreds of safety-critical systems. Without verification tools, these migrations are validated by testing alone—a process that the software engineering community has long recognized as inadequate for safety-critical code. SEMREC provides the missing verification layer: automated, formal, and practical.

Beyond the DoD mandate, the tool has immediate applications in:

- **Safety-critical systems:** Automotive (ISO 26262), aerospace (DO-178C), and medical device software where migration correctness must be assured.
- **LLM-assisted development:** As LLMs are increasingly used for code translation, tools that verify translation correctness become essential infrastructure.
- **Open-source ecosystem:** Major projects (Linux kernel, Firefox, Android) are incrementally adopting Rust; verifying the boundaries between C and Rust code is critical.

10 Conclusion

We have presented SEMREC, a framework for cross-language equivalence verification between C and Rust. The core insight is the *LLVM IR erasure thesis*: existing tools that compare programs through a shared compilation target cannot detect the most important class of migration bugs—divergences rooted in different source-language semantics that compile to identical IR. SEMREC addresses this by preserving source-level semantic distinctions through a shared typed SSA intermediate representation parameterized by semantic configurations, constructing product programs with coercion points at semantic divergence sites, and discharging verification conditions via SMT solving with differential fuzzing as a completeness fallback.

We have formalized the semantic reconciliation framework with 23 definitions, proved product program soundness (Theorem 5.2), divergence witness correctness (Theorems 5.3 and 5.4), three-output exhaustiveness (Theorem 5.5), and coercion insertion completeness (Theorem 5.6), all with explicit assumptions and honest trust boundaries. The integer reconciliation algebra with overflow alignment contracts, the directed ε -approximation with error accumulation for floating-point tolerance, and the angelic nondeterminism treatment of C’s undefined behavior constitute the formal contributions.

On a benchmark of 32 C $\leftrightarrow$ Rust function pairs spanning 10 divergence categories, SEMREC proves equivalence for 16 pairs and produces concrete counterexamples (extracted from Z3 models) for the remaining 16, with zero unknowns. The dominant divergence class is signed integer overflow (12/16 divergences), reflecting C2Rust’s translation of signed operations to Rust’s wrapping arithmetic. The current benchmark is small-scale and consists of focused, few-line functions. Scaling to whole-library analysis of real C2Rust output (e.g., libsodium, zlib) and head-to-head comparison with IR-level tools (Alive2, LLREVE) remain important future work.

We believe this work establishes source-level semantic reconciliation as a promising paradigm for cross-language equivalence verification, and provides the formal and algorithmic foundation for building practical tools that serve the urgent need for verified migration.

References

- [1] V. Astrauskas, P. Müller, F. Poli, and A. J. Summers. Leveraging Rust types for modular specification and verification. In *Proc. OOPSLA*, 2019.
- [2] H. Barbosa, C. W. Barrett, M. Brain, G. Kremer, H. Lachnitt, M. Mann, A. Mohamed, M. Mohamed, A. Niemetz, A. Nötzli, A. Ozdemir, M. Preiner, A. Reynolds, Y. Sheng, C. Tinelli, and Y. Zohar. cvc5: A versatile and industrial-strength SMT solver. In *Proc. TACAS*, 2022.
- [3] G. Barthe, J. M. Crespo, and C. Kunz. Relational verification using product programs. In *Proc. FM*, 2011.
- [4] M. Brain, C. Tinelli, P. Rümmer, and T. Wahl. An automatable formal semantics for IEEE-754 floating-point arithmetic. In *Proc. ARITH*, 2015.
- [5] P. Emre, A. Shankar, and Z. Anderson. C2Rust: Migrating legacy code to Rust. In *Proc. ICSE (Tool Demo)*, 2019.
- [6] Cybersecurity and Infrastructure Security Agency. The case for memory safe roadmaps. Technical report, December 2023.
- [7] E. Clarke, D. Kroening, and F. Lerda. A tool for checking ANSI-C programs. In *Proc. TACAS*, 2004.
- [8] L. de Moura and N. Bjørner. Z3: An efficient SMT solver. In *Proc. TACAS*, 2008.
- [9] X. Denis, J.-H. Jourdan, and C. Marché. Creusot: A foundry for the deductive verification of Rust programs. In *Proc. ICFEM*, 2022.
- [10] M. Eilers, P. Müller, and A. J. Summers. Modular product programs. In *Proc. ESOP*, 2018.
- [11] M. Emre, R. Schroeder, K. Dewey, and B. Hardekopf. Translating C to safer Rust. In *Proc. OOPSLA*, 2021.

- [12] D. Felsing, S. Grebing, V. Klebanov, P. Rümmer, and M. Ulbrich. Automating regression verification. In *Proc. ASE*, 2014.
- [13] A. Fioraldi, D. Maier, H. Eissfeldt, and M. Heuse. AFL++: Combining incremental steps of fuzzing research. In *Proc. WOOT*, 2020.
- [14] Google Security Blog. Memory safety: An update on our progress. Technical report, 2023.
- [15] L. Hadarean, K. Bansal, D. Jovanovic, C. Barrett, and C. Tinelli. A tale of two solvers: Eager and lazy approaches to bit-vectors. In *Proc. CAV*, 2014.
- [16] ISO/IEC 9899:2011. Programming languages — C. International standard, 2011.
- [17] R. Jung, J.-H. Jourdan, R. Krebbers, and D. Dreyer. RustBelt: Securing the foundations of the Rust programming language. In *Proc. POPL*, 2018.
- [18] Amazon Web Services. Kani Rust Verifier. <https://model-checking.github.io/kani/>, 2022.
- [19] A. Lattuada, T. Hance, C. Cho, M. Brun, I. Subasingha, Y. Zhou, J. Howell, B. Parno, and C. Hawblitzel. Verus: Verifying Rust programs using linear ghost types. In *Proc. OOPSLA*, 2023.
- [20] X. Leroy. Formal verification of a realistic compiler. *Communications of the ACM*, 52(7):107–115, 2009.
- [21] Z. Li, J. Wang, M. Sun, and J. C. S. Lui. Detecting cross-language memory management issues in Rust. In *Proc. ESEC/FSE*, 2022.
- [22] LLVM Project. libFuzzer – a library for coverage-guided fuzz testing. <https://llvm.org/docs/LibFuzzer.html>.
- [23] N. P. Lopes, D. Menendez, S. Nagarakatte, and J. Regehr. Provably correct peephole optimizations with Alive. In *Proc. PLDI*, 2015.
- [24] N. P. Lopes, J. Lee, C.-K. Hur, Z. Liu, and J. Regehr. Alive2: Bounded translation validation for LLVM. In *Proc. PLDI*, 2021.
- [25] W. M. McKeeman. Differential testing for software. *Digital Technical Journal*, 10(1):100–107, 1998.
- [26] Microsoft Security Response Center. A proactive approach to more secure code. Blog post, July 2019.
- [27] P. Müller, M. Schwerhoff, and A. J. Summers. Viper: A verification infrastructure for permission-based reasoning. In *Proc. VMCAI*, 2016.
- [28] M. O. Myreen. Verified just-in-time compiler on x86. In *Proc. POPL*, 2010.
- [29] G. C. Necula. Translation validation for an optimizing compiler. In *Proc. PLDI*, 2000.
- [30] R. Pan, A. R. Ibrahimzada, R. Krishna, D. Sanber, B. Cummins, S. Murali, and R. Just. Lost in translation: A study of bugs introduced by large language models while translating code. In *Proc. ICSE*, 2024.

- [31] A. Pnueli, M. Siegel, and E. Singerman. Translation validation. In *Proc. TACAS*, 1998.
- [32] D. A. Ramos and D. Engler. Under-constrained symbolic execution: Correctness checking for real code. In *Proc. USENIX Security*, 2015.
- [33] J. Regehr, Y. Chen, P. Cuoq, E. Eide, C. Ellison, and X. Yang. Test-case reduction for C compiler bugs. In *Proc. PLDI*, 2012.
- [34] The Rust Reference. <https://doc.rust-lang.org/reference/>.
- [35] M. Sousa and I. Dillig. Cartesian Hoare logic for verifying k -safety properties. In *Proc. PLDI*, 2016.
- [36] R. Tate, M. Stepp, Z. Tatlock, and S. Lerner. Equality saturation: A new approach to optimization. In *Proc. POPL*, 2009.
- [37] X. Wang, H. Chen, A. Cheung, Z. Jia, N. Zeldovich, and M. F. Kaashoek. Undefined behavior: What happened to my code? In *Proc. APSys*, 2012.
- [38] White House Office of the National Cyber Director. Back to the building blocks: A path toward secure and measurable software. Technical report, February 2024.
- [39] X. Yang, Y. Chen, E. Eide, and J. Regehr. Finding and understanding bugs in C compilers. In *Proc. PLDI*, 2011.
- [40] H. Zhang, D. Kolouri, Z. Li, and P. Naithani. Crown: Ownership reasoning for C. In *Proc. PLDI*, 2023.
- [41] M. Zhang, H. Zhang, Z. Li, and P. Naithani. Laertes: Ownership-guided C-to-Rust translation. In *Proc. OOPSLA*, 2023.
- [42] L. Zuck, A. Pnueli, and B. Goldberg. VOC: A methodology for the translation validation of optimizing compilers. *Journal of Universal Computer Science*, 9(3):223–247, 2003.